

LAPORAN PRAKTIKUM
PEMROGRAMAN WEB – GIK1JAB3
MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJAX TANPA REFRESH



Alya Mutiya Ningsih Pertiwi

707012500132

D4 SIKC 49-04

PROGRAM STUDI D4 SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

1. Tujuan Praktikum

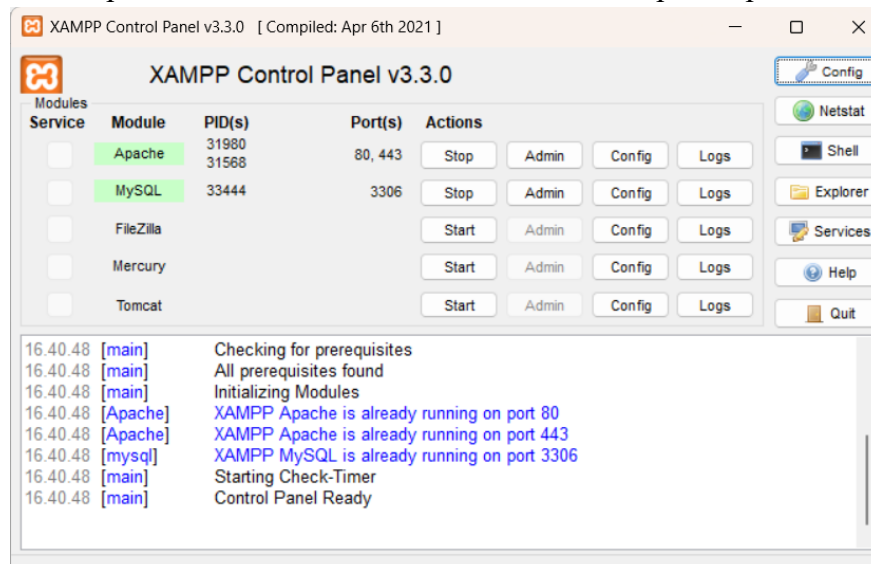
1. Memahami konsep HMVC pada CodeIgniter 3.
2. Mengimplementasikan struktur modular (feature-based).
3. Membuat REST API berbasis JSON.
4. Mengintegrasikan Model dengan REST Controller.
5. Menggunakan AJAX untuk komunikasi asynchronous.
6. Membuat fitur CRUD tanpa refresh halaman.
7. Menguji API menggunakan Postman.
8. Mengintegrasikan frontend dengan REST API.

2. Alat dan Bahan

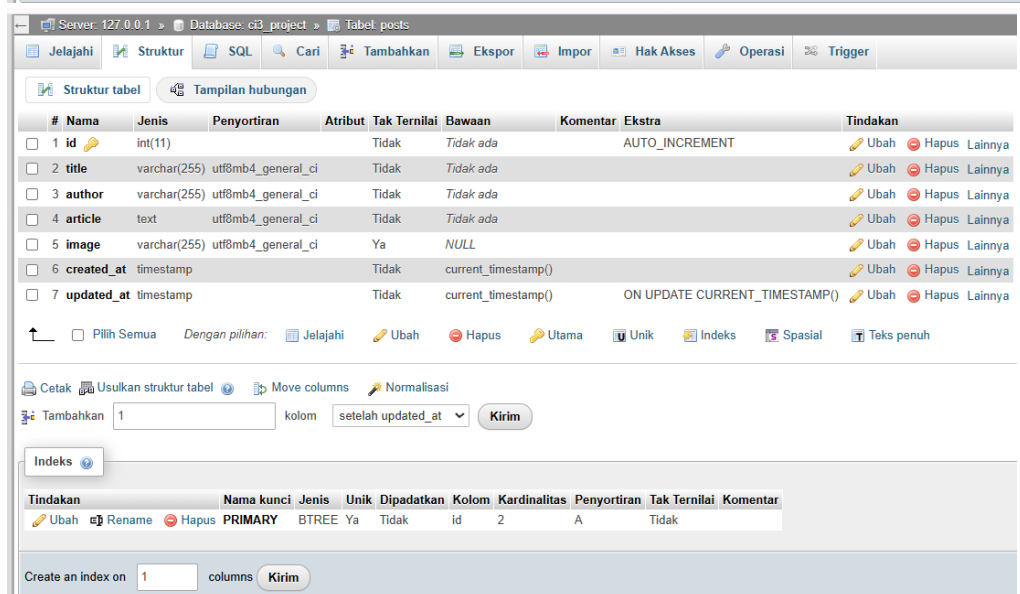
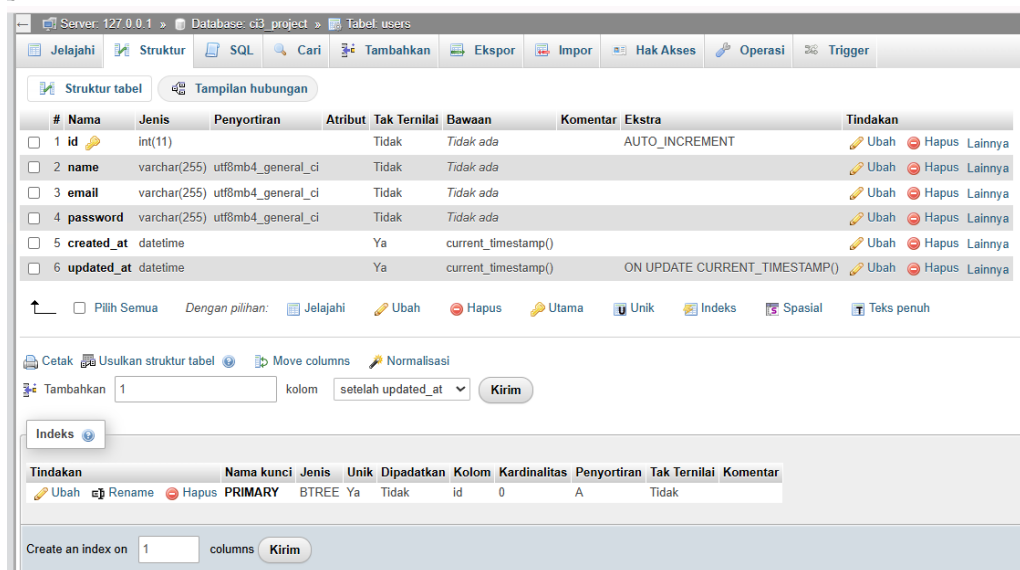
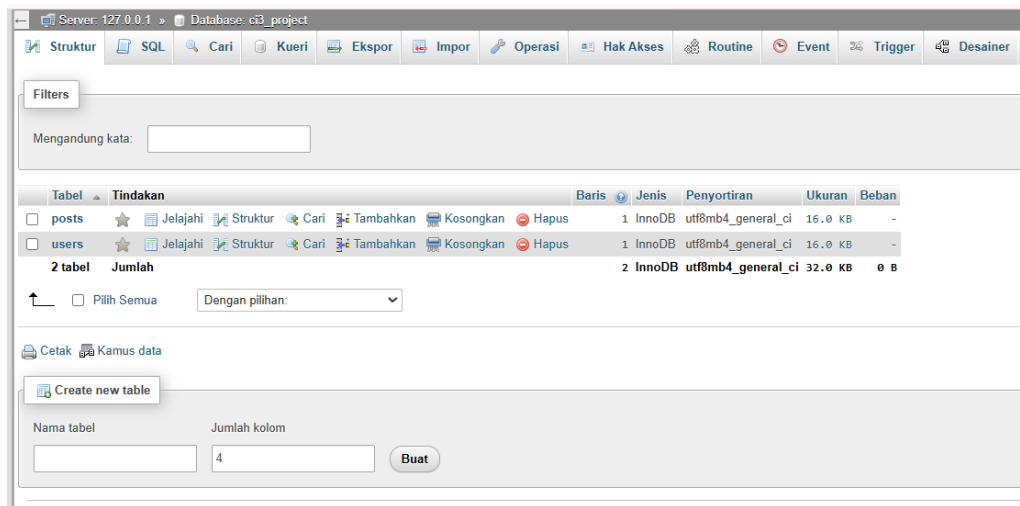
1. XAMPP (Apache & MySQL aktif)
2. PHP 7.x
3. CodeIgniter 3
4. Library HMVC (Modular Extensions – MX)
5. Library REST Server (chriskacerguis)
6. Postman
7. VS Code / Code Editor
8. Browser (Chrome/Firefox)

3. Langkah-langkah Praktikum

1. Buka Aplikasi XAMPP lalu aktifkan menu “start” pada Apache dan MySQL.



2. Masuk pada bagian “Admin” dalam XAMPP pada MySQL, kemudian persiapkan database.



3. Masuk ke bagian “Explorer” di pojok kanan, lalu masuk ke dalam file bernama “htdocs”, kemudian buat folder baru dan pindahkan folder Code Igniter yang telah di buat pada Tugas latihan Pekan-8 pada folder baru tersebut, lalu buat program php pada folder tersebut pada Visual Studio Code.

4. Mengubah code pada application/config/database.php untuk membuat koneksi antara CI3 dengan database MySQL menggunakan Visual Studio Code.



```
1 $active_group = 'default';
2 $query_builder = TRUE;
3
4 $db['default'] = array(
5     'dsn' => '',
6     'hostname' => 'localhost',
7     'username' => 'root',
8     'password' => '',
9     'database' => 'ci3_project',
10    'dbdriver' => 'mysqli',
11    'dbprefix' => '',
12    'pconnect' => FALSE,
13    'db_debug' => (ENVIRONMENT !== 'production'
14    n'), 'cache_on' => FALSE,
15    'cachedir' => '',
16    'char_set' => 'utf8',
17    'dbcollat' => 'utf8_general_ci',
18    'swap_pre' => '',
19    'encrypt' => FALSE,
20    'compress' => FALSE,
21    'stricton' => FALSE,
22    'failover' => array(),
23    'save_queries' => TRUE
24 );
```

5. Tambahkan struktur HMVC pada folder application/modules. Buat folder Crudjs untuk fitur CRUD AJAX di dalam modules, kemudian buat folder controllers lalu pada Folder Controllers buat file Crudjs.php. Isi program Crudjs.php:

```

1 <?php
2 defined('BASEPATH') OR exit('No direct script access allowed');
3
4 class Crudjs extends CI_Controller {
5
6     public function __construct() {
7         parent::__construct();
8         $this->load->model('crudjs_model');
9         $this->load->helper(array('url', 'form'));
10        $this->load->library(array('form_validation', 'upload', 'session'));
11    }
12
13    public function debug() {
14        $debug_info = array();
15
16        $debug_info['database'] = array(
17            'connected' => $this->db->conn_id ? 'Yes' : 'No',
18            'database' => $this->db->database
19        );
20
21        $debug_info['table_exists'] = $this->db->table_exists('posts') ? 'Yes' : 'No';
22
23        $query = $this->db->get('posts', 1);
24        $debug_info['table_accessible'] = $query ? 'Yes' : 'No';
25        $debug_info['record count'] = $this->db->count_all('posts');
26
27        $upload_path = './uploads/posts/';
28        $debug_info['upload_folder'] = array(
29            'exists' => is_dir($upload_path) ? 'Yes' : 'No',
30            'writable' => is_writable($upload_path) ? 'Yes' : 'No'
31        );
32
33        $debug_info['sample_records'] = array();
34        try {
35            $records = $this->crudjs_model->get_all();
36            if (empty($records)) {
37                // Show first record as sample
38                $first = $records[0];
39                $debug_info['sample_records'] = array(
40                    'count' => count($records),
41                    'first_record' => $first
42                );
43            }
44        } catch (Exception $e) {
45            $debug_info['sample_records'][]['error'] = $e->getMessage();
46        }
47
48        echo 'qres' . json_encode($debug_info, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES) . "\n";
49    }
50
51    public function index() {
52        $data['title'] = 'CRUD with AJAX';
53        $this->load->view('crudjs/index', $data);
54    }
55
56    public function get_all() {
57        ob_clean();
58        header('Content-Type: application/json; charset=utf-8');
59
60        try {
61            $records = $this->crudjs_model->get_all();
62
63            $response = array(
64                'status' => 'success',
65                'data' => $records
66            );
67
68            echo json_encode($response, JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
69        } catch (Exception $e) {
70            echo json_encode(array(
71                'status' => 'error',
72                'message' => 'Failed to load records: ' . $e->getMessage()
73            ));
74        }
75        exit;
76    }
77
78    public function get_record($id) {
79        ob_clean();
80        header('Content-type: application/json; charset=utf-8');
81
82        try {
83            if (!($this->crudjs_model->exists($id))) {
84                echo json_encode(array(
85                    'status' => 'error',
86                    'message' => 'Record not found'
87                ));
88                exit;
89            }
90
91            $record = $this->crudjs_model->get_by_id($id);
92
93            echo json_encode(array(
94                'status' => 'success',
95                'data' => $record
96            ), JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
97        } catch (Exception $e) {
98            echo json_encode(array(
99                'status' => 'error',
100                'message' => 'Failed to load record: ' . $e->getMessage()
101            ));
102        }
103        exit;
104    }
105
106    public function store() {
107        ob_clean();
108        header('Content-Type: application/json; charset=utf-8');
109
110        try {
111            $this->form_validation->set_rules('title', 'Title', 'required|max_length[255]');
112            $this->form_validation->set_rules('author', 'Author', 'required|max_length[255]');
113            $this->form_validation->set_rules('article', 'Article', 'required');
114
115            if ($this->form_validation->run() == FALSE) {
116                echo json_encode(array(
117                    'status' => 'error',
118                    'message' => 'Validation error: ' . strip_tags(validation_errors())
119                ));
120                exit;
121            }
122
123            $data = array(
124                'title' => $this->input->post('title'),
125                'author' => $this->input->post('author'),
126                'article' => $this->input->post('article'),
127                'created_at' => date('Y-m-d H:i:s'),
128                'updated_at' => date('Y-m-d H:i:s')
129            );
130
131            if (empty($_FILES['image']['name'])) {
132                $upload_result = $this->upload_image();
133
134                if ($upload_result['status']) {
135                    $data['image'] = 'posts/' . $upload_result['file_name'];
136                } else {
137                    echo json_encode(array(
138                        'status' => 'error',
139                        'message' => $upload_result['error']
140                    ));
141                    exit;
142                }
143            }
144
145            $record_id = $this->crudjs_model->insert($data);
146
147            if ($record_id) {
148                $new_record = $this->crudjs_model->get_by_id($record_id);
149                echo json_encode(array(
150                    'status' => 'success',
151                    'message' => 'Record created successfully!',
152                    'data' => $new_record
153                ), JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
154            } else {
155                echo json_encode(array(
156                    'status' => 'error',
157                    'message' => 'Failed to create record'
158                ));
159            }
160        } catch (Exception $e) {
161            echo json_encode(array(
162                'status' => 'error',
163                'message' => 'Exception error: ' . $e->getMessage()
164            ));
165        }
166        exit;
167    }

```

```

1 public function update($id) {
2     ob_clean();
3     header('Content-Type: application/json; charset=utf-8');
4
5     try {
6         if (!$this->crudjs_model->exists($id)) {
7             echo json_encode(array(
8                 'status' => 'error',
9                 'message' => 'Record not found'
10            ));
11             exit;
12         }
13
14         $this->form_validation->set_rules('title', 'Title', 'max_length[255]');
15         $this->form_validation->set_rules('author', 'Author', 'max_length[255]');
16
17         if ($this->form_validation->run() === FALSE) {
18             echo json_encode(array(
19                 'status' => 'error',
20                 'message' => 'Validation error: ' . strip_tags(validation_errors())
21            ));
22             exit;
23         }
24
25         $data = array(
26             'updated at' => date('Y-m-d H:i:s')
27         );
28
29         if ($this->input->post('title')) {
30             $data['title'] = $this->input->post('title');
31         }
32         if ($this->input->post('author')) {
33             $data['author'] = $this->input->post('author');
34         }
35         if ($this->input->post('article')) {
36             $data['article'] = $this->input->post('article');
37         }
38
39         if (!empty($_FILES['image']['name'])) {
40             $old_record = $this->crudjs_model->get_by_id($id);
41             if ($old_record->image) {
42                 $old_image_path = './uploads/' . $old_record->image;
43                 if (file_exists($old_image_path)) {
44                     unlink($old_image_path);
45                 }
46             }
47
48             $upload_result = $this->upload_image();
49
50             if ($upload_result['status']) {
51                 $data['image'] = 'posts/' . $upload_result['file_name'];
52             } else {
53                 echo json_encode(array(
54                     'status' => 'error',
55                     'message' => $upload_result['error']
56                 ));
57                 exit;
58             }
59         }
60
61         $result = $this->crudjs_model->update($id, $data);
62
63         if ($result) {
64             $updated_record = $this->crudjs_model->get_by_id($id);
65             echo json_encode(array(
66                 'status' => 'success',
67                 'message' => 'Record updated successfully!',
68                 'data' => $updated_record
69            ));
70             $this->output->send_header('X-XSS-Protection: 1; mode=block');
71             echo json_encode(array(
72                 'status' => 'error',
73                 'message' => 'Failed to update record'
74            ));
75         }
76     } catch (Exception $e) {
77         echo json_encode(array(
78             'status' => 'error',
79             'message' => 'Exception error: ' . $e->getMessage()
80         ));
81     }
82     exit;
83 }
84
85 public function delete($id) {
86     ob_clean();
87     header('Content-Type: application/json; charset=utf-8');
88
89     try {
90         $record = $this->crudjs_model->get_by_id($id);
91
92         if (!$record) {
93             echo json_encode(array(
94                 'status' => 'error',
95                 'message' => 'Record not found'
96            ));
97             exit;
98         }
99
100         if ($record->image) {
101             $image_path = './uploads/' . $record->image;
102             if (file_exists($image_path)) {
103                 unlink($image_path);
104             }
105         }
106
107         $result = $this->crudjs_model->delete($id);
108
109         if ($result) {
110             echo json_encode(array(
111                 'status' => 'success',
112                 'message' => 'Record deleted successfully!'
113            ));
114         } else {
115             echo json_encode(array(
116                 'status' => 'error',
117                 'message' => 'Failed to delete record'
118            ));
119         }
120     } catch (Exception $e) {
121         echo json_encode(array(
122             'status' => 'error',
123             'message' => 'Exception error: ' . $e->getMessage()
124         ));
125     }
126     exit;
127 }
128
129 public function test() {
130     ob_clean();
131     header('Content-Type: application/json; charset=utf-8');
132     echo json_encode(array('status' => 'success', 'message' => 'API is working'));
133     exit;
134 }
135
136 private function _upload_image() {
137     $upload_path = './uploads/posts/';
138
139     if (!is_dir($upload_path)) {
140         mkdir($upload_path, 0777, true);
141     }
142
143     $original_name = $_FILES['image']['name'];
144     $sanitized_name = preg_replace('/[A-Za-z0-9_]/', '_', $original_name);
145     $file_name = time() . '_' . $sanitized_name;
146
147     $config['upload_path'] = $upload_path;
148     $config['allowed_types'] = 'jpg|png|jpeg';
149     $config['max_size'] = 2048; // KB
150     $config['file_name'] = $file_name;
151     $config['overwrite'] = FALSE;
152
153     $this->upload->initialize($config);
154
155     if ($this->upload->do_upload('image')) {
156         $upload_data = $this->upload->data();
157         return array(
158             'status' => TRUE,
159             'file_name' => $upload_data['file_name']
160         );
161     } else {
162         return array(
163             'status' => FALSE,
164             'error' => $this->upload->display_errors('', '')
165         );
166     }
167 }
168 }

```

6. Tambahkan folder baru bernama models pada application\modules\Crudjs kemudian buat file baru bernama Crudjs_model.php. Isi program Crudjs_model.php:

```
1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class Crudjs_model extends CI_Model {
5
6      private $table = 'posts';
7
8      public function __construct() {
9          parent::__construct();
10         $this->load->database();
11     }
12
13     public function get_all() {
14         $query = $this->db->get($this->table);
15         $records = $query->result();
16
17         foreach ($records as $record) {
18             if ($record->image) {
19                 $record->image_url = 'http://localhost/ci3_project/uploads/posts/' . $record->image;
20             } else {
21                 $record->image_url = null;
22             }
23         }
24
25         return $records;
26     }
27
28     public function get_by_id($id) {
29         $query = $this->db->get_where($this->table, array('id' => $id));
30         $record = $query->row();
31
32         if ($record && $record->image) {
33             $record->image_url = 'http://localhost/ci3_project/uploads/posts/' . $record->image;
34         } else if ($record) {
35             $record->image_url = null;
36         }
37
38         return $record;
39     }
40
41     public function insert($data) {
42         $this->db->insert($this->table, $data);
43         return $this->db->insert_id();
44     }
45
46     public function update($id, $data) {
47         $this->db->where('id', $id);
48         return $this->db->update($this->table, $data);
49     }
50
51     public function delete($id) {
52         $this->db->where('id', $id);
53         return $this->db->delete($this->table);
54     }
55
56     public function exists($id) {
57         $query = $this->db->get_where($this->table, array('id' => $id));
58         return $query->num_rows() > 0;
59     }
60 }
```

7. Tambahkan folder baru bernama views pada application\modules\Crudjs kemudian buat file baru bernama header.php. Isi program header.php:

```

1  <!DOCTYPE html>
2  <html Lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title><?php echo isset($title) ? $title : 'CRUD with AJAX'; ?> - CI3 CRUD</title>
7  </head>
8  <body>
9      * {
10         margin: 0;
11         padding: 0;
12         box-sizing: border-box;
13     }
14     body {
15         font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
16         background-color: #f5f5f5;
17         line-height: 1.6;
18     }
19     .container {
20         max-width: 1200px;
21         margin: 0 auto;
22         padding: 20px;
23     }
24     header {
25         background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
26         color: white;
27         padding: 30px 0;
28         margin-bottom: 30px;
29         box-shadow: 0 4px 6px rgba(0,0,0,0.1);
30     }
31     header h1 {
32         text-align: center;
33         font-size: 2.5em;
34         margin-bottom: 10px;
35     }
36     header p {
37         text-align: center;
38         font-size: 1.1em;
39         opacity: 0.9;
40     }
41     .nav {
42         background: white;
43         padding: 15px 0;
44         margin-bottom: 30px;
45         box-shadow: 0 2px 4px rgba(0,0,0,0.1);
46         border-radius: 8px;
47     }
48     .nav ul {
49         list-style: none;
50         display: flex;
51         justify-content: center;
52         gap: 20px;
53     }
54     .nav a {
55         color: #667eea;
56         text-decoration: none;
57         padding: 10px 20px;
58         border-radius: 5px;
59         transition: all 0.3s;
60         font-weight: 500;
61     }
62     .nav a:hover {
63         background: #667eea;
64         color: white;
65     }
66     .alert {
67         padding: 15px 20px;
68         border-radius: 8px;
69         margin-bottom: 20px;
70         animation: slideDown 0.3s ease;
71         display: none;
72     }
73     .alert.show {
74         display: block;
75     }
76     @keyframes slideDown {
77         from {
78             opacity: 0;
79             transform: translateY(-10px);
80         }
81         to {
82             opacity: 1;
83             transform: translateY(0);
84         }
85     }
86     .alert-success {
87         background-color: #d4edda;
88         border-left: 4px solid #28a745;
89         color: #155724;
90     }

```



```

1  .alert-error {
2      background-color: #f8d7da;
3      border-left: 4px solid #dc3545;
4      color: #721c24;
5  }
6
7  .btn {
8      display: inline-block;
9      padding: 10px 20px;
10     text-decoration: none;
11     border-radius: 5px;
12     cursor: pointer;
13     border: none;
14     font-size: 14px;
15     font-weight: 500;
16     transition: all 0.3s;
17 }
18
19 .btn-primary {
20     background: linear-gradient(135deg, #667eea 0%, #764ba2 10
21 0%);color: white;
22 }
23
24 .btn-primary:hover {
25     transform: translateY(-2px);
26     box-shadow: 0 4px 8px rgba(102, 126, 234, 0.4);
27 }
28
29 .btn-success {
30     background-color: #28a745;
31     color: white;
32 }
33
34 .btn-success:hover {
35     background-color: #218838;
36     transform: translateY(-2px);
37     box-shadow: 0 4px 8px rgba(40, 167, 69, 0.4);
38 }
39
40 .btn-danger {
41     background-color: #dc3545;
42     color: white;
43     border: none;
44 }
45
46 .btn-danger:hover {
47     background-color: #c82333;
48 }
49
50 .btn-warning {
51     background-color: #ffc107;
52     color: #212529;
53 }
54
55 .btn-warning:hover {
56     background-color: #e0a800;
57 }
58
59 .btn-info {
60     background-color: #17a2b8;
61     color: white;
62 }
63
64 .btn-info:hover {
65     background-color: #138496;
66 }
67
68 .btn-secondary {
69     background-color: #6c757d;
70     color: white;
71 }
72
73 .btn-secondary:hover {
74     background-color: #5a6268;
75 }
76
77 .card {
78     background: white;
79     border-radius: 10px;
80     box-shadow: 0 2px 10px rgba(0,0,0,0.1);
81     padding: 30px;
82     margin-bottom: 20px;
83 }
84
85 .form-group {
86     margin-bottom: 20px;
87 }
88
89 .form-group label {
90     display: block;
91     margin-bottom: 8px;
92     font-weight: 600;
93     color: #333;
94 }
95
96 .form-group input[type="text"],
97 .form-group textarea,
98 .form-group input[type="file"] {
99     width: 100%;
100    padding: 12px;
101    border: 2px solid #e0e0e0;
102    border-radius: 6px;
103    font-size: 14px;
104    transition: border-color 0.3s;
105    font-family: inherit;
106 }

```

```

1  .form-group input[type="text"]:focus,
2  .form-group textarea:focus {
3      outline: none;
4      border-color: #667eea;
5  }
6
7  .form-group textarea {
8      min-height: 150px;
9      resize: vertical;
10 }
11
12 table {
13     width: 100%;
14     border-collapse: collapse;
15     background: white;
16     border-radius: 8px;
17     overflow: hidden;
18     box-shadow: 0 2px 10px rgba(0,0,0,0.1);
19 }
20
21 table thead {
22     background: linear-gradient(135deg, #667eea 0%, #764ba2 10
23 0%);color: white;
24 }
25
26 table th {
27     padding: 15px;
28     text-align: left;
29     font-weight: 600;
30 }
31
32 table td {
33     padding: 15px;
34     border-bottom: 1px solid #e0e0e0;
35 }
36
37 table tbody tr:hover {
38     background-color: #f8f9fa;
39 }
40
41 table tbody tr:last-child td {
42     border-bottom: none;
43 }
44
45 .actions {
46     display: flex;
47     gap: 5px;
48 }
49
50 .post-image {
51     max-width: 100px;
52     height: auto;
53     border-radius: 5px;
54     box-shadow: 0 2px 4px rgba(0,0,0,0.1);
55 }
56
57 .post-detail-image {
58     max-width: 500px;
59     width: 100%;
60     height: auto;
61     border-radius: 10px;
62     box-shadow: 0 4px 8px rgba(0,0,0,0.1);
63     margin: 20px 0;
64 }
65
66 .text-center {
67     text-align: center;
68 }
69
70 .article-preview {
71     max-width: 200px;
72     overflow: hidden;
73     text-overflow: ellipsis;
74     white-space: nowrap;
75 }
76
77 footer {
78     background: #333;
79     color: white;
80     text-align: center;
81     padding: 20px 0;
82     margin-top: 50px;
83 }
84
85 .back-link {
86     display: inline-block;
87     margin-bottom: 20px;
88 }
89
90 .modal {
91     display: none;
92     position: fixed;
93     z-index: 1000;
94     left: 0;
95     top: 0;
96     width: 100%;
97     height: 100%;
98     background-color: rgba(0, 0, 0, 0.5);
99     animation: fadeIn 0.3s ease;
100 }

```

```

1      @keyframes fadeIn {
2          from { opacity: 0; }
3          to { opacity: 1; }
4      }
5
6      .modal.show {
7          display: flex;
8          align-items: center;
9          justify-content: center;
10     }
11
12     .modal-content {
13         background-color: white;
14         padding: 30px;
15         border-radius: 10px;
16         width: 90%;
17         max-width: 600px;
18         box-shadow: 0 4px 20px rgba(0,0,0,0.3);
19         animation: slideup 0.3s ease;
20     }
21
22     @keyframes slideup {
23         from {
24             opacity: 0;
25             transform: translateY(50px);
26         }
27         to {
28             opacity: 1;
29             transform: translateY(0);
30         }
31     }
32
33     .modal-header {
34         display: flex;
35         justify-content: space-between;
36         align-items: center;
37         margin-bottom: 20px;
38         padding-bottom: 15px;
39         border-bottom: 2px solid #e0e0e0;
40     }
41
42     .modal-header h2 {
43         margin: 0;
44         color: #333;
45     }
46
47     .close {
48         font-size: 28px;
49         font-weight: bold;
50         color: #999;
51         cursor: pointer;
52         transition: color 0.3s;
53     }
54
55     .close:hover {
56         color: #333;
57     }
58
59     .modal-body {
60         margin-bottom: 20px;
61     }
62
63     .modal-footer {
64         display: flex;
65         gap: 10px;
66         justify-content: flex-end;
67     }
68
69     .loading {
70         display: none;
71         text-align: center;
72         padding: 20px;
73         color: #6677ee;
74     }
75
76     .loading-show {
77         display: block;
78     }
79
80     .spinner {
81         border: 4px solid #faf3f3;
82         border-top: 4px solid #6677ee;
83         border-radius: 50%;
84         width: 40px;
85         height: 40px;
86         animation: spin 1s linear infinite;
87         margin: 0 auto 10px;
88     }
89
90     @keyframes spin {
91         0% { transform: rotate(0deg); }
92         100% { transform: rotate(360deg); }
93     }
94
95     .header-actions {
96         display: flex;
97         justify-content: space-between;
98         align-items: center;
99         margin-bottom: 20px;
100    }
101
102    .header-actions h2 {
103        margin: 0;
104    }
105
106    small {
107        display: block;
108        margin-top: 5px;
109        color: #666;
110    }
111
112    </style>
113    </head>
114    <body>
115        <header>
116            <div class="container">
117                <h1> CRUD with AJAX </h1>
118                <p>CodeIgniter 3 CRUD Application - No Page Refresh</p>
119            </div>
120        </header>
121
122        <div class="container">
123            <div class="nav">
124                <ul>
125                    <li><a href="#<php echo base_url(); ?>">Home</a></li>
126                    <li><a href="#<php echo base_url('crudjs')>">CRUD with AJAX</a></li>
127                </ul>
128            </div>
129
130            <div id="alertContainer">
131                <div id="successAlert" class="alert alert-success"></div>
132                <div id="errorAlert" class="alert alert-error"></div>
133            </div>

```

8. Pada folder views application\modules\Crudjs\views buat file baru bernama index.php. Isi program index.php:

```
1 <?php $this->load->view('crudjs/header'); ?>
2
3 <div class="card">
4     <div class="header-actions">
5         <h2>All Records</h2>
6         <button type="button" class="btn btn-success" onclick="openCreateModal()">+ Create New Record</button>
7     </div>
8
9     <table>
10         <thead>
11             <tr>
12                 <th style="width: 50px;">ID</th>
13                 <th style="width: 100px;">Image</th>
14                 <th>Title</th>
15                 <th>Author</th>
16                 <th style="width: 250px;">Article Preview</th>
17                 <th style="width: 200px;">Actions</th>
18             </tr>
19         </thead>
20         <tbody>
21             <tr>
22                 <td colspan="6" style="text-align: center; padding: 40px;">Loading...</td>
23             </tr>
24         </tbody>
25     </table>
26 </div>
27
28 <?php $this->load->view('crudjs/footer'); ?>
```

9. Pada folder views application\modules\Crudjs\views buat file baru bernama footer.php. Isi program footer.php:

```

1 </div> <!-- End container -->
2
3 <footer>
4   <div class="container">
5     <p>COPY: <php echo date("Y"); >?> CRUD with AJAX - Built with CodeIgniter 3</p>
6   </div>
7 </footer>
8
9 <div id="formModal" class="modal">
10   <div class="modal-content">
11     <div class="modal-header">
12       <h2 id="modalTitle">Create New Record</h2>
13       <span class="close" onclick="closeModal()">&times;</span>
14     </div>
15     <div class="modal-body">
16       <div class="loading" id="loading">
17         <div class="spinner"></div>
18         <p>Processing...</p>
19       </div>
20       <form id="crudForm" enctype="multipart/form-data">
21         <input type="hidden" id="recordId" name="recordId">
22
23         <div class="form-group">
24           <label for="title">Title <span style="color: red;">*</span></label>
25           <input type="text"
26             name="title"
27             id="title"
28             placeholder="Enter title"
29             required>
30         </div>
31
32         <div class="form-group">
33           <label for="author">Author <span style="color: red;">*</span></label>
34           <input type="text"
35             name="author"
36             id="author"
37             placeholder="Enter author name"
38             required>
39         </div>
40
41         <div class="form-group">
42           <label for="article">Article <span style="color: red;">*</span></label>
43           <textarea name="article"
44             id="article"
45             placeholder="Write your content here..."
46             required></textarea>
47         </div>
48
49         <div class="form-group">
50           <label for="image">Image</label>
51           <div id="currentImageContainer" style="display: none; margin-bottom: 10px;">
52             <p style="margin-bottom: 10px; font-weight: 500;">Current Image:</p>
53             <img id="currentImage"
54               alt="Current image"
55               style="max-width: 200px; border-radius: 8px; box-shadow: 0 2px 4px rgba(0,0,0,0.1);">
56           </div>
57           <input type="file"
58             name="image"
59             id="image"
60             accept="image/jpeg,image/jpg,image/png">
61           <small>Allowed formats: JPG, JPEG, PNG. Maximum size: 2MB</small>
62         </div>
63       </form>
64     </div>
65     <div class="modal-footer">
66       <button type="button" class="btn btn-secondary" onclick="closeModal()">Cancel</button>
67       <button type="button" class="btn btn-success" onclick="submitForm()">Save</button>
68     </div>
69 </div>
70 </div>
71
72 <div id="detailModal" class="modal">
73   <div class="modal-content">
74     <div class="modal-header">
75       <h2 id="detailTitle">Record Details</h2>
76       <span class="close" onclick="closeDetailModal()">&times;</span>
77     </div>
78     <div class="modal-body">
79       <div class="loading" id="detailLoading">
80         <div class="spinner"></div>
81         <p>Loading...</p>
82       </div>
83       <div id="detailContent" style="display: none;">
84         <div id="detailData"></div>
85       </div>
86     </div>
87     <div class="modal-footer">
88       <button type="button" class="btn btn-warning" onclick="editFromDetail()">Edit</button>
89       <button type="button" class="btn btn-danger" onclick="deleteFromDetail()">Delete</button>
90       <button type="button" class="btn btn-secondary" onclick="closeDetailModal()">Close</button>
91     </div>
92 </div>
93 </div>
94
95 <script>
96   const baseUrl = '<php echo base_url(); >?';
97   let isEditing = false;
98   let currentRecordId = null;
99
100   document.addEventListener('DOMContentLoaded', function() {
101     console.log('CRUDjs Initialized - Base URL:', baseUrl);
102     loadRecords();
103   }, false);
104
105   function loadRecords() {
106     const url = baseUrl + 'crudis/get_all';
107     console.log('Loading records from:', url);
108
109     fetch(url, {
110       method: 'GET',
111       headers: {
112         'Content-Type': 'application/json'
113       }
114     })
115     .then(response => {
116       if (!response.ok) {
117         throw new Error("HTTP error! status: " + response.status);
118       }
119       return response.json();
120     })
121     .then(data => {
122       console.log('Records loaded:', data);
123       if (data.status === 'success') {
124         renderTable(data.data);
125       } else if (data.status === 'error') {
126         showError(data.message || 'Failed to load records');
127       }
128     })
129     .catch(error => {
130       showError('Failed to load records: ' + error.message);
131       console.error('Load records error:', error);
132     });
133   }
134
135   function renderTable(records) {
136     const tbody = document.querySelector('table tbody');
137     if (!tbody) return;
138
139     tbody.innerHTML = '';
140
141     if (records.length === 0) {
142       tbody.innerHTML = '<tr><td colspan="6" style="text-align: center; padding: 40px;">No records found. Create your first record</td></tr>';
143       return;
144     }

```

```

1 records.forEach(record => {
2   const row = document.createElement('tr');
3   const imageUrl = record.image_url ? `` : `<span style="color: #999;">No image</span>`;
4   const articlePreview = record.article.substring(0, 100) + (record.article.length > 100 ? '...' : '');
5
6   row.innerHTML = `
7     <td>${record.id}</td>
8     <td>${imageUrl}</td>
9     <td>${record.title}</td>
10    <td>${record.author}</td>
11    <td>
12      <div class="article-preview">${escapeHtml(articlePreview)}</div>
13    </td>
14    <td>
15      <div class="actions">
16        <button type="button" class="btn btn-info" onclick="viewRecord(${record.id})">View</button>
17        <button type="button" class="btn btn-warning" onclick="editRecord(${record.id})">Edit</button>
18        <button type="button" class="btn btn-danger" onclick="deleteRecord(${record.id})">Delete</button>
19      </div>
20    </td>
21  `;
22  tbody.appendChild(row);
23  });
24 }
25
26 function openCreateModal() {
27   isEditing = false;
28   currentRecordId = null;
29   document.getElementById('modalTitle').textContent = 'Create New Record';
30   document.getElementById('crudForm').reset();
31   document.getElementById('recordId').value = '';
32   document.getElementById('currentImageContainer').style.display = 'none';
33   document.getElementById('formModal').classList.add('show');
34 }
35
36 function editRecord(id) {
37   isEditing = true;
38   currentRecordId = id;
39   document.getElementById('modalTitle').textContent = 'Edit Record';
40   document.getElementById('recordId').value = id;
41
42   const url = baseUrl + 'crudjs/get_record/' + id;
43   console.log('Loading record from:', url);
44
45   fetch(url, {
46     method: 'GET',
47     headers: {
48       'Content-Type': 'application/json'
49     }
50   })
51   .then(response => {
52     if (!response.ok) {
53       throw new Error('HTTP error! status: ' + response.status);
54     }
55     return response.json();
56   })
57   .then(data => {
58     console.log('Record loaded:', data);
59     if (data.status === 'success') {
60       const record = data.data;
61       document.getElementById('title').value = record.title;
62       document.getElementById('author').value = record.author;
63       document.getElementById('article').value = record.article;
64
65       if (record.image_url) {
66         document.getElementById('currentImage').src = record.image_url;
67         document.getElementById('currentImageContainer').style.display = 'block';
68       } else {
69         document.getElementById('currentImageContainer').style.display = 'none';
70       }
71
72       document.getElementById('formModal').classList.add('show');
73     } else {
74       showError(data.message || 'Failed to load record');
75     }
76   })
77   .catch(error => {
78     showError('Failed to load record: ' + error.message);
79     console.error('Load record error:', error);
80   });
81 }
82
83 function submitForm() {
84   const form = document.getElementById('crudForm');
85   const formData = new FormData(form);
86
87   const recordId = document.getElementById('recordId').value;
88   const url = (isEditing ? baseUrl + 'crudjs/update/' + recordId : baseUrl + 'crudjs/store');
89
90   console.log('Submitting form to:', url, 'Edit mode:', isEditing);
91   document.getElementById('loading').classList.add('show');
92
93   fetch(url, {
94     method: 'POST',
95     body: formData
96   })
97   .then(response => {
98     if (!response.ok) {
99       throw new Error('HTTP error! status: ' + response.status);
100     }
101     return response.json();
102   })
103   .then(data => {
104     console.log('Form submitted, response:', data);
105     document.getElementById('loading').classList.remove('show');
106
107     if (data.status === 'success') {
108       showSuccess(data.message);
109       closeModal();
110       loadRecords();
111     } else {
112       showError(data.message || 'Operation failed');
113     }
114   })
115   .catch(error => {
116     console.error('Submit error:', error);
117     document.getElementById('loading').classList.remove('show');
118     showError('An error occurred: ' + error.message);
119   });
120 }
121
122 function viewRecord(id) {
123   console.log('Viewing record:', id);
124   document.getElementById('detailModal').classList.add('show');
125   document.getElementById('detailLoading').classList.add('show');
126   document.getElementById('detailContent').style.display = 'none';
127
128   const url = baseUrl + 'crudjs/get_record/' + id;
129   console.log('Loading detail from:', url);
130
131   fetch(url, {
132     method: 'GET',
133     headers: {
134       'Content-Type': 'application/json'
135     }
136   })
137   .then(response => {
138     if (!response.ok) {
139       throw new Error('HTTP error! status: ' + response.status);
140     }
141     return response.json();
142   })
143   .then(data => {
144     console.log('Detail loaded:', data);
145     if (data.status === 'success') {
146       const record = data.data;
147       currentRecordId = record.id;
148       renderDetailView(record);
149     } else {
150       showError(data.message || 'Failed to load record details');
151       document.getElementById('detailLoading').classList.remove('show');
152     }
153   })
154   .catch(error => {
155     console.error('Load detail error:', error);
156     showError('Failed to load record: ' + error.message);
157     document.getElementById('detailLoading').classList.remove('show');
158   });
159 }

```

```

1      function renderDetailView(record) {
2          const detailData = document.getElementById('detailData');
3          let detailHtml = `
4              <h3>${escapeHtml(record.title)}</h3>
5              <hr style="margin: 20px 0; border: none; border-top: 2px solid #000000;" />
6              <p><strong>Author:</strong> ${escapeHtml(record.author)}</p>
7              <p><strong>Created:</strong> ${new Date(record.created_at).toLocaleString()}</p>
8          `;
9
10         if (record.updated_at && record.updated_at !== record.created_at) {
11             detailHtml += `<p><strong>Updated:</strong> ${new Date(record.updated_at).toLocaleString()}</p>`;
12         }
13
14         if (record.image_url) {
15             detailHtml += ``;
16         }
17
18         detailHtml += `
19             <div style="margin-top: 30px;">
20                 <h4 style="margin-bottom: 15px; color: #333;">Content</h4>
21                 <div style="line-height: 1.8; color: #555; white-space: pre-wrap;">
22                     ${escapeHtml(record.article)}
23                 </div>
24             </div>
25         `;
26
27         detailData.innerHTML = detailHtml;
28         document.getElementById('detailLoading').classList.remove('show');
29         document.getElementById('detailContent').style.display = 'block';
30         document.getElementById('detailTitle').textContent = 'Record Details';
31     }
32
33     function deleteRecord(id) {
34         if (!confirm('Are you sure you want to delete this record?')) {
35             return;
36         }
37
38         console.log('Deleting record:', id);
39         const url = baseUrl + 'crudjs/delete/' + id;
40         console.log('Delete URL:', url);
41
42         fetch(url, {
43             method: 'POST',
44             headers: {
45                 'Content-Type': 'application/json'
46             }
47         })
48         .then(response => {
49             if (!response.ok) {
50                 throw new Error('HTTP error! status: ' + response.status);
51             }
52             return response.json();
53         })
54         .then(data => {
55             console.log('Delete response:', data);
56             if (data.status === 'success') {
57                 showSuccess(data.message);
58                 loadRecords();
59             } else {
60                 showError(data.message || 'Failed to delete record');
61             }
62         })
63         .catch(error => {
64             console.error('Delete error:', error);
65             showError('Failed to delete record: ' + error.message);
66         });
67     }
68
69     function deleteFromDetail() {
70         if (!currentRecordId) return;
71
72         if (!confirm('Are you sure you want to delete this record?')) {
73             return;
74         }
75
76         deleteRecord(currentRecordId);
77         closeDetailModal();
78     }
79
80     function editFromDetail() {
81         if (!currentRecordId) return;
82
83         closeDetailModal();
84         editRecord(currentRecordId);
85     }
86
87     function closeModal() {
88         document.getElementById('formModal').classList.remove('show');
89     }
90
91     function closeDetailModal() {
92         document.getElementById('detailModal').classList.remove('show');
93     }
94
95     function showSuccess(message) {
96         console.log('Success:', message);
97         const alert = document.getElementById('successAlert');
98         alert.textContent = '✓ ' + message;
99         alert.classList.add('show');
100         setTimeout(() => {
101             alert.classList.remove('show');
102         }, 4000);
103     }
104
105     function showError(message) {
106         console.error('Error:', message);
107         const alert = document.getElementById('errorAlert');
108         alert.textContent = 'X ' + message;
109         alert.classList.add('show');
110         setTimeout(() => {
111             alert.classList.remove('show');
112         }, 6000);
113     }
114
115     function escapeHtml(text) {
116         const div = document.createElement('div');
117         div.textContent = text;
118         return div.innerHTML;
119     }
120
121     window.onclick = function(event) {
122         const formModal = document.getElementById('formModal');
123         const detailModal = document.getElementById('detailModal');
124
125         if (event.target === formModal) {
126             closeModal();
127         }
128         if (event.target === detailModal) {
129             closeDetailModal();
130         }
131     }
132 </script>
133 </body>
134 </html>

```

10. Buat file Jwt.php pada folder application\libraries. Isi program Jwt.php:


```

1 <?php
2 defined('BASEPATH') OR exit('No direct script access allowed');
3
4 class Jwt {
5     private $secret_key = 'your_secret_key_change_this_in_production';
6     private $algorithm = 'HS256';
7     private $expiration = 604800; // 7 days in seconds
8
9     public function __construct()
10     {
11         $this->secret_key = config_item('jwt_secret_key') ? $this->secret_key;
12         $this->expiration = config_item('jwt_expiration') ? $this->expiration;
13     }
14
15     /**
16      * Create JWT token
17      * @param array $payload Data to encode in token
18      * @return string JWT token
19      */
20     public function create($payload = array())
21     {
22         $header = array(
23             'typ' => 'JWT',
24             'alg' => $this->algorithm
25         );
26
27         $payload['iat'] = time();
28         $payload['exp'] = time() + $this->expiration;
29
30         $header_encoded = $this->base64_url_encode(json_encode($header));
31         $payload_encoded = $this->base64_url_encode(json_encode($payload));
32
33         $signature = hash_hmac(
34             'sha256',
35             $header_encoded . '.' . $payload_encoded,
36             $this->secret_key,
37             true
38         );
39         $signature_encoded = $this->base64_url_encode($signature);
40
41         return $header_encoded . '.' . $payload_encoded . '.' . $signature_encoded;
42     }
43
44     /**
45      * Verify and decode JWT token
46      * @param string $token JWT token to verify
47      * @return object|false Decoded payload or false if invalid
48      */
49     public function verify($token = '')
50     {
51         if (empty($token)) {
52             return false;
53         }
54
55         $parts = explode('.', $token);
56         if (count($parts) != 3) {
57             return false;
58         }
59
60         list($header_encoded, $payload_encoded, $signature_encoded) = $parts;
61
62         $signature = hash_hmac(
63             'sha256',
64             $header_encoded . '.' . $payload_encoded,
65             $this->secret_key,
66             true
67         );
68         $signature_computed = $this->base64_url_encode($signature);
69
70         if ($signature_computed != $signature_encoded) {
71             return false;
72         }
73
74         $payload = json_decode($this->base64_url_decode($payload_encoded));
75
76         if (isset($payload->exp) && $payload->exp < time()) {
77             return false;
78         }
79
80         return $payload;
81     }
82
83     /**
84      * Get token from request header
85      * @return string|false token or false if not found
86      */
87     public function get_token_from_request()
88     {
89         $auth_header = null;
90
91         if (function_exists('apache_request_headers')) {
92             $headers = apache_request_headers();
93             $auth_header = isset($headers['Authorization']) ? $headers['Authorization'] : null;
94         }
95
96         if (!$auth_header && isset($_SERVER['HTTP_AUTHORIZATION'])) {
97             $auth_header = $_SERVER['HTTP_AUTHORIZATION'];
98         }
99
100         if (!$auth_header && isset($_SERVER['REDIRECT_HTTP_AUTHORIZATION'])) {
101             $auth_header = $_SERVER['REDIRECT_HTTP_AUTHORIZATION'];
102         }
103
104         if (!$auth_header) {
105             return false;
106         }
107
108         if (preg_match('/Bearer\s(\S+)/', $auth_header, $matches)) {
109             return $matches[1];
110         }
111
112         return false;
113     }
114
115     private function base64_url_encode($input)
116     {
117         return str_replace(
118             array('+', '/', '='),
119             array('-', '_', ''),
120             base64_encode($input)
121         );
122     }
123
124     private function base64_url_decode($input)
125     {
126         $remainder = strlen($input) % 4;
127         if ($remainder) {
128             $input .= str_repeat('=', 4 - $remainder);
129         }
130
131         return base64_decode(
132             str_replace(
133                 array('-', '_'),
134                 array('+', '/'),
135                 $input
136             )
137         );
138     }
139 }
140
141 )

```

11. Buat folder Api pada application\modules, kemudian buat folder controllers di dalamnya lalu buat file bernama Auth.php. Isi program Auth.php:

```

1 <?php
2 defined('BASEPATH') OR exit('No direct script access allowed');
3
4 class Auth extends MX_Controller {
5
6     public function __construct()
7     {
8         parent::__construct();
9         $this->load->model('User_model');
10        $this->load->library('jwt');
11        $this->load->library('form_validation');
12    }
13
14    public function register()
15    {
16        header('Content-Type: application/json; charset=utf-8');
17        ob_clean();
18
19        try {
20            $input = json_decode(file_get_contents('php://input'), true);
21
22            if (empty($input['name']) || empty($input['email']) || empty($input['password'])) {
23                http_response_code(400);
24                echo json_encode([
25                    'status' => false,
26                    'message' => 'Name, email, and password are required'
27                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
28                exit;
29            }
30
31            if (filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
32                http_response_code(400);
33                echo json_encode([
34                    'status' => false,
35                    'message' => 'Invalid email format'
36                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
37                exit;
38            }
39
40            if ($this->User_model->email_exists($input['email'])) {
41                http_response_code(400);
42                echo json_encode([
43                    'status' => false,
44                    'message' => 'Email already registered'
45                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
46                exit;
47            }
48
49            if (strlen($input['password']) < 6) {
50                http_response_code(400);
51                echo json_encode([
52                    'status' => false,
53                    'message' => 'Password must be at least 6 characters'
54                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
55                exit;
56            }
57
58            $user_data = [
59                'name' => htmlspecialchars($input['name']),
60                'email' => strtolower($input['email']),
61                'password' => password_hash($input['password'], PASSWORD_BCRYPT)
62            ];
63
64            $user_id = $this->User_model->create($user_data);
65
66            if ($user_id) {
67                http_response_code(500);
68                echo json_encode([
69                    'status' => false,
70                    'message' => 'Failed to create user'
71                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
72                exit;
73            }
74
75            $user = $this->User_model->get_by_id($user_id);
76
77            $token = $this->jwt->create([
78                'id' => $user->id,
79                'name' => $user->name,
80                'email' => $user->email
81            ]);
82
83            http_response_code(201);
84            echo json_encode([
85                'status' => true,
86                'message' => 'User registered successfully',
87                'data' => [
88                    'id' => $user->id,
89                    'name' => $user->name,
90                    'email' => $user->email,
91                    'created_at' => $user->created_at
92                ],
93                'access_token' => $token,
94                'token_type' => 'Bearer'
95            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
96            exit;
97        } catch (Exception $e) {
98            http_response_code(500);
99            echo json_encode([
100                'status' => false,
101                'message' => 'Server error: ' . $e->getMessage()
102            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
103            exit;
104        }
105    }
106
107    public function login()
108    {
109        header('Content-Type: application/json; charset=utf-8');
110        ob_clean();
111
112        try {
113            $input = json_decode(file_get_contents('php://input'), true);
114
115            if (empty($input['email']) || empty($input['password'])) {
116                http_response_code(400);
117                echo json_encode([
118                    'status' => false,
119                    'message' => 'Email and password are required'
120                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
121                exit;
122            }
123
124            $user = $this->User_model->get_by_email(strtolower($input['email']));
125
126            if (!$user) {
127                http_response_code(401);
128                echo json_encode([
129                    'status' => false,
130                    'message' => 'Invalid credentials'
131                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
132                exit;
133            }
134
135            if (password_verify($input['password'], $user->password)) {
136                http_response_code(401);
137                echo json_encode([
138                    'status' => false,
139                    'message' => 'Invalid credentials'
140                ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
141                exit;
142            }
143
144            $token = $this->jwt->create([
145                'id' => $user->id,
146                'name' => $user->name,
147                'email' => $user->email
148            ]);
149
150

```

```

1      http_response_code(200);
2      echo json_encode([
3          'status' => true,
4          'message' => 'login successful',
5          'data' => [
6              'id' => $user->id,
7              'name' => $user->name,
8              'email' => $user->email
9          ],
10         'access_token' => $token,
11         'token_type' => 'Bearer'
12     ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
13     exit;
14
15 } catch (Exception $e) {
16     http_response_code(500);
17     echo json_encode([
18         'status' => false,
19         'message' => 'Server error: ' . $e->getMessage()
20     ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
21     exit;
22 }
23
24
25 public function logout()
26 {
27     header('Content-Type: application/json; charset=utf-8');
28     ob_clean();
29
30     try {
31         $token = $this->jwt->get_token_from_request();
32
33         if (!$token) {
34             http_response_code(401);
35             echo json_encode([
36                 'status' => false,
37                 'message' => 'No token provided'
38             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
39             exit;
40         }
41
42         $decoded = $this->jwt->verify($token);
43
44         if (!$decoded) {
45             http_response_code(401);
46             echo json_encode([
47                 'status' => false,
48                 'message' => 'Invalid token'
49             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
50             exit;
51         }
52
53         $user = $this->User_model->get_by_id($decoded->id);
54
55         if (!$user) {
56             http_response_code(404);
57             echo json_encode([
58                 'status' => false,
59                 'message' => 'User not found'
60             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
61             exit;
62         }
63
64         http_response_code(200);
65         echo json_encode([
66             'status' => true,
67             'message' => 'logout successful'
68         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
69         exit;
70     } catch (Exception $e) {
71         http_response_code(500);
72         echo json_encode([
73             'status' => false,
74             'message' => 'Server error: ' . $e->getMessage()
75         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
76         exit;
77     }
78 }
79
80 public function me()
81 {
82     header('Content-Type: application/json; charset=utf-8');
83     ob_clean();
84
85     try {
86         $token = $this->jwt->get_token_from_request();
87
88         if (!$token) {
89             http_response_code(401);
90             echo json_encode([
91                 'status' => false,
92                 'message' => 'No token provided'
93             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
94             exit;
95         }
96
97         $decoded = $this->jwt->verify($token);
98
99         if (!$decoded) {
100             http_response_code(401);
101             echo json_encode([
102                 'status' => false,
103                 'message' => 'Invalid or expired token'
104             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
105             exit;
106         }
107
108         $user = $this->User_model->get_by_id($decoded->id);
109
110         if (!$user) {
111             http_response_code(404);
112             echo json_encode([
113                 'status' => false,
114                 'message' => 'User not found'
115             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
116             exit;
117         }
118
119         http_response_code(200);
120         echo json_encode([
121             'status' => true,
122             'message' => 'User retrieved successfully',
123             'data' => [
124                 'id' => $user->id,
125                 'name' => $user->name,
126                 'email' => $user->email,
127                 'created_at' => $user->created_at,
128                 'updated_at' => $user->updated_at
129             ]
130         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
131         exit;
132     } catch (Exception $e) {
133         http_response_code(500);
134         echo json_encode([
135             'status' => false,
136             'message' => 'Server error: ' . $e->getMessage()
137         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
138         exit;
139     }
140 }
141
142 }
143

```

12. Pada folder `application\modules\Api\controllers`, buat file bernama `Post.php`. Isi program `Post.php`:

```

1 <?php
2 defined('BASEPATH') OR exit('No direct script access allowed');
3
4 class Post extends MX_Controller {
5
6     public function __construct()
7     {
8         parent::__construct();
9         $this->load->model('Post_model');
10        $this->load->library('jet');
11        $this->load->library('upload');
12    }
13
14    private function parse_multipart_form()
15    {
16        $content_type = $_SERVER['CONTENT_TYPE'] ?? '';
17        $input = [];
18
19        if (strpos($content_type, 'multipart/form-data') !== false) {
20            $boundary = '';
21            if (preg_match('/boundary=([^\s;]+)/', $content_type, $matches)) {
22                $boundary = trim($matches[1], '"');
23            }
24
25            if (!$boundary) {
26                return [];
27            }
28
29            $raw = file_get_contents('php://input');
30            $parts = preg_split('/' . preg_quote($boundary) . '/', $raw);
31
32            foreach ($parts as $part) {
33                if (empty(trim($part)) || $part === "---\n" || $part === "---") {
34                    continue;
35                }
36
37                $split = preg_split("/\n\n\r\n/", trim($part), 2);
38                if (count($split) !== 2) {
39                    continue;
40                }
41
42                $headers = $split[0];
43                $content = rtrim($split[1], "\n\r");
44
45                if (preg_match('/name=("[^\s"]*)"/', $headers, $matches)) {
46                    $name = $matches[1];
47
48                    if (preg_match('/filename=("[^\s"]*)"/', $headers, $filename_matches)) {
49                        $file_content_type = 'application/octet-stream';
50                        if (preg_match('/content-type:([^\s;]+)/', $headers, $type_matches)) {
51                            $file_content_type = trim($type_matches[1]);
52                        }
53
54                        $input[$name] = [
55                            'name' => $filename_matches[1],
56                            'type' => $file_content_type,
57                            'content' => $content
58                        ];
59                    } else {
60                        $input[$name] = $content;
61                    }
62                }
63            }
64
65            return $input;
66        }
67
68        if (strpos($content_type, 'application/x-www-form-urlencoded') !== false) {
69            $parse_str(file_get_contents('php://input'), $input);
70            return $input ?? [];
71        }
72
73        return array_merge($_POST ?? [], $_FILES ?? []);
74    }
75
76    private function get_image_url($image_filename)
77    {
78        if (empty($image_filename)) {
79            return null;
80        }
81        return 'http://localhost/c13-project/uploads/posts/' . $image_filename;
82    }
83
84    public function handle($id = null)
85    {
86        $method = $_SERVER['REQUEST_METHOD'];
87
88        if ($method === 'GET') {
89            if ($id) {
90                $this->show($id);
91            } else {
92                $this->index();
93            }
94        } elseif ($method === 'POST') {
95            $this->store();
96        } elseif ($method === 'PUT') {
97            $this->update($id);
98        } elseif ($method === 'DELETE') {
99            $this->delete($id);
100        } else {
101            http_response_code(405);
102            header('Content-Type: application/json; charset=utf-8');
103            echo json_encode([
104                'status' => false,
105                'message' => 'Method not allowed'
106            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
107            exit;
108        }
109    }
110
111    public function index()
112    {
113        header('Content-Type: application/json; charset=utf-8');
114        ob_clean();
115
116        try {
117            $page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;
118            $per_page = isset($_GET['per_page']) ? max(1, min(100, (int)$_GET['per_page'])) : 10;
119            $offset = ($page - 1) * $per_page;
120
121            $posts = $this->Post_model->get_all($per_page, $offset);
122            $total = $this->Post_model->count_all();
123
124            foreach ($posts as $post) {
125                if (!empty($post->image)) {
126                    $post->image_url = $this->get_image_url($post->image);
127                } else {
128                    $post->image_url = null;
129                }
130            }
131
132            http_response_code(200);
133            echo json_encode([
134                'status' => true,
135                'message' => 'Posts retrieved successfully',
136                'data' => $posts,
137                'pagination' => [
138                    'total' => (int)$total,
139                    'per_page' => $per_page,
140                    'current_page' => $page,
141                    'last_page' => ceil($total / $per_page)
142                ]
143            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
144            exit;
145        } catch (Exception $e) {
146            http_response_code(500);
147            echo json_encode([
148                'status' => false,
149                'message' => 'Server error: ' . $e->getMessage()
150            ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
151            exit;
152        }
153    }
154
155    }
156

```

```

1 public function show($id)
2 {
3     header('Content-Type: application/json; charset=utf-8');
4     ob_clean();
5
6     try {
7         if (!is_numeric($id)) {
8             http_response_code(400);
9             echo json_encode([
10                 'status' => false,
11                 'message' => 'Invalid post ID'
12             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
13             exit;
14         }
15
16         $post = $this->Post_model->get_by_id($id);
17
18         if (!$post) {
19             http_response_code(404);
20             echo json_encode([
21                 'status' => false,
22                 'message' => 'Post not found'
23             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
24             exit;
25         }
26
27         if (!empty($post->image)) {
28             $post->image_url = $this->get_image_url($post->image);
29         } else {
30             $post->image_url = null;
31         }
32
33         http_response_code(200);
34         echo json_encode([
35             'status' => true,
36             'message' => 'Post retrieved successfully',
37             'data' => $post
38         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
39         exit;
40     } catch (Exception $e) {
41         http_response_code(500);
42         echo json_encode([
43             'status' => false,
44             'message' => 'Server error: ' . $e->getMessage()
45         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
46         exit;
47     }
48 }
49
50 public function store()
51 {
52     header('Content-Type: application/json; charset=utf-8');
53     ob_clean();
54
55     try {
56         $token = $this->Jwt->get_token_from_request();
57
58         if (!$token) {
59             http_response_code(401);
60             echo json_encode([
61                 'status' => false,
62                 'message' => 'Unauthorized: No token provided'
63             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
64             exit;
65         }
66
67         $decoded = $this->Jwt->verify($token);
68
69         if (!$decoded) {
70             http_response_code(401);
71             echo json_encode([
72                 'status' => false,
73                 'message' => 'Unauthorized: Invalid or expired token'
74             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
75             exit;
76         }
77
78         $input = json_decode(file_get_contents('php://input'), true);
79
80         if (!isset($input['title']) || !isset($input['article'])) {
81             http_response_code(400);
82             echo json_encode([
83                 'status' => false,
84                 'message' => 'Title and article are required'
85             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
86             exit;
87         }
88
89         $post_data = [
90             'title' => htmlspecialchars($input['title']),
91             'author' => htmlspecialchars($input['author'] ?? 'Anonymous'),
92             'article' => htmlspecialchars($input['article'])
93         ];
94
95         $image_name = '';
96         if (isset($_FILES['image'])['name']) {
97             $image_name = $this->upload_image();
98             if (!$image_name) {
99                 http_response_code(400);
100                 echo json_encode([
101                     'status' => false,
102                     'message' => 'Image upload failed'
103                 ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
104                 exit;
105             }
106             $post_data['image'] = $image_name;
107         }
108
109         $post_id = $this->Post_model->create($post_data);
110
111         if (!$post_id) {
112             http_response_code(500);
113             echo json_encode([
114                 'status' => false,
115                 'message' => 'Failed to create post'
116             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
117             exit;
118         }
119
120         $post = $this->Post_model->get_by_id($post_id);
121
122         if (!empty($post->image)) {
123             $post->image_url = $this->get_image_url($post->image);
124         } else {
125             $post->image_url = null;
126         }
127
128         http_response_code(201);
129         echo json_encode([
130             'status' => true,
131             'message' => 'Post created successfully',
132             'data' => $post
133         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
134         exit;
135     } catch (Exception $e) {
136         http_response_code(500);
137         echo json_encode([
138             'status' => false,
139             'message' => 'Server error: ' . $e->getMessage()
140         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
141         exit;
142     }
143 }
144
145 }

```

```

1 public function update($id)
2 {
3     header('Content-Type: application/json; charset=utf-8');
4     ob_clean();
5
6     try {
7         $token = $this->jwt->get_token_from_request();
8
9         if (!$token) {
10             http_response_code(401);
11             echo json_encode([
12                 'status' => false,
13                 'message' => 'Unauthorized: No token provided'
14             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
15             exit;
16         }
17
18         $decoded = $this->jwt->verify($token);
19
20         if (!$decoded) {
21             http_response_code(401);
22             echo json_encode([
23                 'status' => false,
24                 'message' => 'Unauthorized: Invalid or expired token'
25             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
26             exit;
27         }
28
29         if (!is_numeric($id)) {
30             http_response_code(400);
31             echo json_encode([
32                 'status' => false,
33                 'message' => 'Invalid post ID'
34             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
35             exit;
36         }
37
38         if (!$this->Post_model->exists($id)) {
39             http_response_code(404);
40             echo json_encode([
41                 'status' => false,
42                 'message' => 'Post not found'
43             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
44             exit;
45         }
46
47         $input = json_decode(file_get_contents('php://input'), true);
48         if (!$input) {
49             $input = $this->parse_multipart_form();
50
51             if (empty($input['image']) && is_array($input['image']) && isset($input['image']['content'])) {
52                 $FILES['image'] = [
53                     'name' => $input['image']['name'] ?? 'image-' . time() . '.png',
54                     'type' => $input['image']['type'] ?? 'image/png',
55                     'tmp_name' => $this->save_temp_file($input['image']['content']),
56                     'error' => 0,
57                     'size' => strlen($input['image']['content'])
58                 ];
59             }
60         }
61
62         if (empty($input['title']) || empty($input['article'])) {
63             http_response_code(400);
64             echo json_encode([
65                 'status' => false,
66                 'message' => 'Title and article are required'
67             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
68             exit;
69         }
70
71         $post_data = [
72             'title' => htmlspecialchars($input['title']),
73             'author' => htmlspecialchars($input['author'] ?? 'Anonymous'),
74             'article' => htmlspecialchars($input['article'])
75         ];
76
77         if (empty($FILES['image']['name']) || (isset($input['image']) && is_array($input['image']) && !empty($input['image']['name']))) {
78             $old_post = $this->Post_model->get_by_id($id);
79
80             if (empty($old_post->image)) {
81                 $old_file = FCPATH . 'uploads/posts/' . $old_post->image;
82                 if (file_exists($old_file)) {
83                     unlink($old_file);
84                 }
85             }
86
87             $image_name = $this->upload_image();
88             if ($image_name) {
89                 $post_data['image'] = $image_name;
90             }
91         }
92
93         $result = $this->Post_model->update($id, $post_data);
94
95         if (!$result) {
96             http_response_code(500);
97             echo json_encode([
98                 'status' => false,
99                 'message' => 'Failed to update post'
100             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
101             exit;
102         }
103
104         $post = $this->Post_model->get_by_id($id);
105
106         if (empty($post->image)) {
107             $post->image_url = $this->get_image_url($post->image);
108         } else {
109             $post->image_url = null;
110         }
111
112         http_response_code(200);
113         echo json_encode([
114             'status' => true,
115             'message' => 'Post updated successfully',
116             'data' => $post
117         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
118         exit;
119     } catch (Exception $e) {
120         http_response_code(500);
121         echo json_encode([
122             'status' => false,
123             'message' => 'Server error: ' . $e->getMessage()
124         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
125         exit;
126     }
127 }
128
129 public function delete($id)
130 {
131     header('Content-Type: application/json; charset=utf-8');
132     ob_clean();
133
134     try {
135         $token = $this->jwt->get_token_from_request();
136
137         if (!$token) {
138             http_response_code(401);
139             echo json_encode([
140                 'status' => false,
141                 'message' => 'Unauthorized: No token provided'
142             ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
143             exit;
144         }
145     }

```



```

1      $decoded = $this->jwt->verify($token);
2
3      if (!$decoded) {
4          http_response_code(401);
5          echo json_encode([
6              'status' => false,
7              'message' => 'Unauthorized: Invalid or expired token'
8          ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
9          exit;
10     }
11
12     if (!is_numeric($id)) {
13         http_response_code(400);
14         echo json_encode([
15             'status' => false,
16             'message' => 'Invalid post ID'
17         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
18         exit;
19     }
20
21     $post = $this->Post_model->get_by_id($id);
22
23     if (!$post) {
24         http_response_code(404);
25         echo json_encode([
26             'status' => false,
27             'message' => 'Post not found'
28         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
29         exit;
30     }
31
32     if (!empty($post->image)) {
33         $file = FCPATH . 'uploads/posts/' . $post->image;
34         if (file_exists($file)) {
35             unlink($file);
36         }
37     }
38
39     $result = $this->Post_model->delete($id);
40
41     if (!$result) {
42         http_response_code(500);
43         echo json_encode([
44             'status' => false,
45             'message' => 'Failed to delete post'
46         ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
47         exit;
48     }
49
50     http_response_code(200);
51     echo json_encode([
52         'status' => true,
53         'message' => 'Post deleted successfully'
54     ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
55     exit;
56 } catch (Exception $e) {
57     http_response_code(500);
58     echo json_encode([
59         'status' => false,
60         'message' => 'Server error: ' . $e->getMessage()
61     ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
62     exit;
63 }
64 }
65
66 private function _save_temp_file($content)
67 {
68     $temp_path = sys_get_temp_dir() . '/ci3_upload_' . uniqid() . '.tm
69 p';
70     file_put_contents($temp_path, $content);
71     return $temp_path;
72 }
73
74 private function _upload_image()
75 {
76     $config['upload_path'] = FCPATH . 'uploads/posts/';
77     $config['allowed_types'] = 'gif|jpg|png|jpeg';
78     $config['max_size'] = 2048; // 2MB
79     $config['file_name'] = 'post_' . time() . '_' . uniqid();
80
81     if (!is_dir($config['upload_path'])) {
82         mkdir($config['upload_path'], 0755, true);
83     }
84
85     $this->upload->initialize($config);
86
87     if (!$this->upload->do_upload('image')) {
88         return false;
89     }
90
91     $upload_data = $this->upload->data();
92     return $upload_data['file_name'];
93 }
94 }

```

13. Pada folder `application\modules\Api`, buat folder baru bernama `models` lalu buat file bernama `Post_model.php` pada folder tersebut. Isi program `Post_model.php`:

```

1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class Post_model extends CI_Model {
5
6      private $table = 'posts';
7
8      public function __construct()
9      {
10         parent::__construct();
11         $this->load->database();
12     }
13
14     public function get_all($limit = 10, $offset = 0)
15     {
16         $query = $this->db
17             ->select('id, title, author, article, image, created_at, updated_at')
18             ->limit($limit, $offset)
19             ->order_by('created_at', 'DESC')
20             ->get($this->table);
21         return $query->result();
22     }
23
24     public function count_all()
25     {
26         return $this->db->count_all($this->table);
27     }
28
29     public function get_by_id($id)
30     {
31         $query = $this->db
32             ->select('id, title, author, article, image, created_at, updated_at')
33             ->get_where($this->table, array('id' => $id));
34         return $query->row();
35     }
36
37     public function create($data)
38     {
39         $data['created_at'] = date('Y-m-d H:i:s');
40         $data['updated_at'] = date('Y-m-d H:i:s');
41
42         $this->db->insert($this->table, $data);
43         return $this->db->insert_id();
44     }
45
46     public function update($id, $data)
47     {
48         $data['updated_at'] = date('Y-m-d H:i:s');
49         $this->db->where('id', $id);
50         return $this->db->update($this->table, $data);
51     }
52
53     public function delete($id)
54     {
55         $this->db->where('id', $id);
56         return $this->db->delete($this->table);
57     }
58
59     public function exists($id)
60     {
61         $query = $this->db->get_where($this->table, array('id' => $id));
62         return $query->num_rows() > 0;
63     }
64
65     public function search($keyword, $limit = 10, $offset = 0)
66     {
67         $this->db->like('title', $keyword);
68         $this->db->or_like('description', $keyword);
69         $this->db->limit($limit, $offset);
70         $this->db->order_by('created_at', 'DESC');
71         $query = $this->db->get($this->table);
72         return $query->result();
73     }
74
75     public function get_by_user($user_id, $limit = 10, $offset = 0)
76     {
77         $query = $this->db
78             ->select('id, title, author, article, image, created_at, updated_at')
79             ->where('user_id', $user_id)
80             ->limit($limit, $offset)
81             ->order_by('created_at', 'DESC')
82             ->get($this->table);
83         return $query->result();
84     }
85 }

```

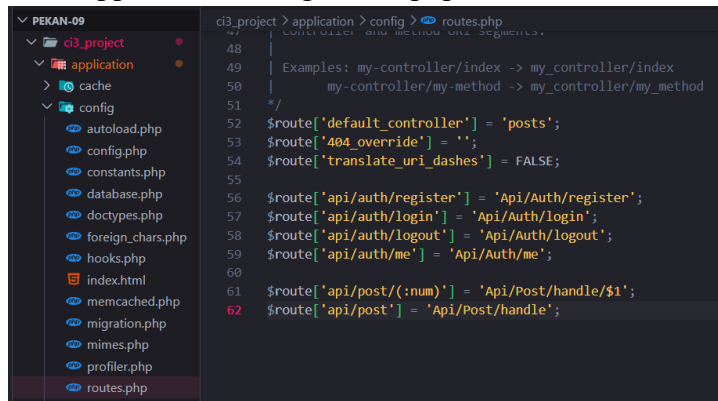
14. Pada folder `application\modules\Api\models`, buat file bernama `User_model.php`.
Isi program `User_model.php`:

```

1  <?php
2  defined('BASEPATH') OR exit('No direct script access allowed');
3
4  class User_model extends CI_Model {
5
6      private $table = 'users';
7
8      public function __construct()
9      {
10         parent::__construct();
11         $this->load->database();
12     }
13
14     public function get_all()
15     {
16         $query = $this->db->get($this->table);
17         return $query->result();
18     }
19
20     public function get_by_id($id)
21     {
22         $query = $this->db->get_where($this->table, array('id' => $id));
23         return $query->row();
24     }
25
26     public function get_by_email($email)
27     {
28         $query = $this->db->get_where($this->table, array('email' => $email));
29         return $query->row();
30     }
31
32     public function create($data)
33     {
34         $data['created_at'] = date('Y-m-d H:i:s');
35         $data['updated_at'] = date('Y-m-d H:i:s');
36
37         $this->db->insert($this->table, $data);
38         return $this->db->insert_id();
39     }
40
41     public function update($id, $data)
42     {
43         $data['updated_at'] = date('Y-m-d H:i:s');
44         $this->db->where('id', $id);
45         return $this->db->update($this->table, $data);
46     }
47
48     public function delete($id)
49     {
50         $this->db->where('id', $id);
51         return $this->db->delete($this->table);
52     }
53
54     public function email_exists($email)
55     {
56         $query = $this->db->get_where($this->table, array('email' => $email));
57         return $query->num_rows() > 0;
58     }
59
60     public function exists($id)
61     {
62         $query = $this->db->get_where($this->table, array('id' => $id));
63         return $query->num_rows() > 0;
64     }
65 }

```

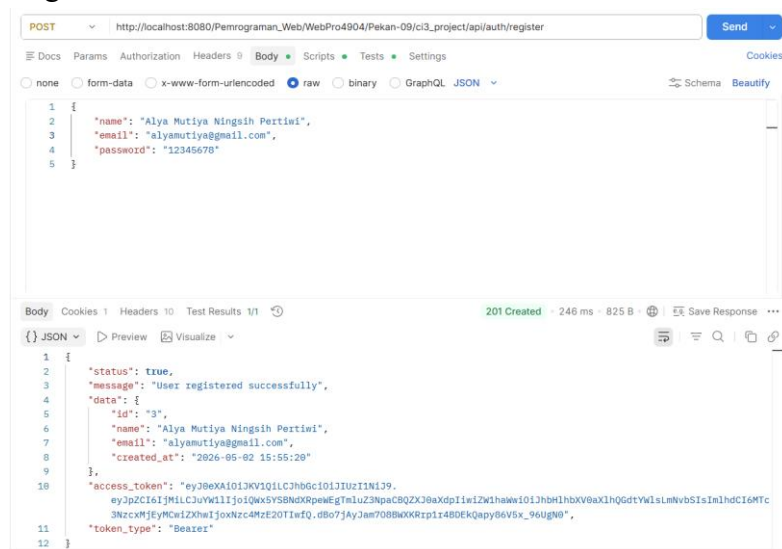
15. Buka `application/config/routes.php` kemudian tambahkan program berikut.



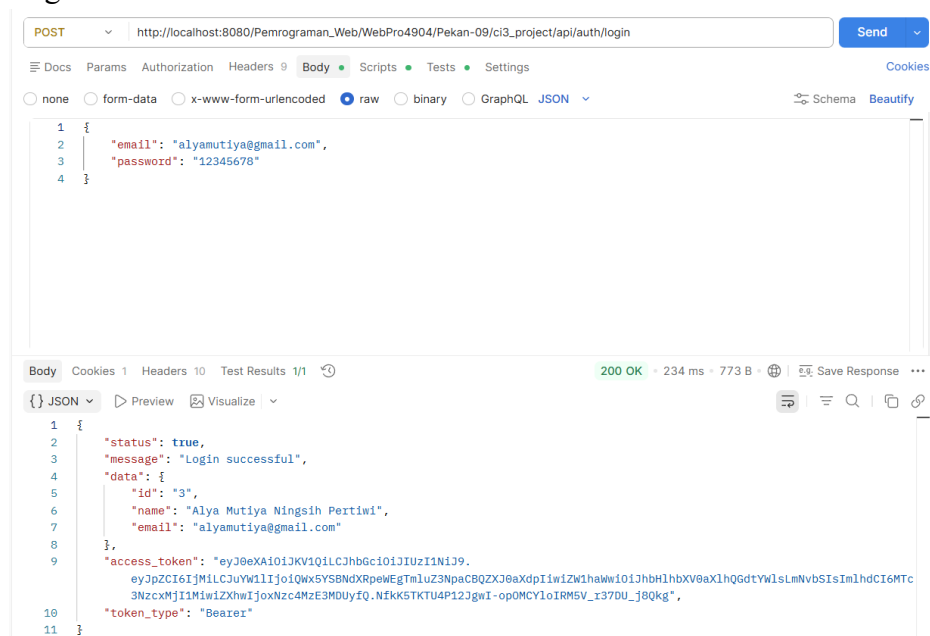
16. Buat folder uploads untuk menyediakan tempat penyimpanan file yang di upload oleh pengguna.

17. Jalankan Aplikasi Restfull API dengan Postman.

a. Register



b. Login



c. Token keamanan login

The screenshot shows the Swagger UI interface for a REST API. The top bar includes a "POST" method, a dropdown menu, and a "Send" button. The URL is `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/auth/login`. The "Auth Type" is set to "Bearer Token". The "Token" field displays `2JgwI-opOMCYIoIRM5V_r37DU_j8Qkg`. Below the token, a message states: "The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization." The bottom section shows the response body in JSON format, indicating a successful login with a 200 OK status, 177 ms response time, and 773 B body size. The JSON response includes a status of true, a success message, user data (id, name, email), an access token, and the token type (Bearer).

```
{
  "status": true,
  "message": "Login successful",
  "data": {
    "id": "3",
    "name": "Alya Mutiya Ningsih Pertiwi",
    "email": "alyamutiya@gmail.com"
  },
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpZCI6IjM1LCJ1Ijo1QWx5YSBNdXRpeWVgTmluZ3NpaCBQZXJ0aXdpIiwiaW1haWwIjo1JHh1bHbXV0aXlhGdtW1sLmNvbSIsIm1hdCI6MTc3NzcxMjQ3NCwiZWhwIjo5Nzc4MzE3Mjc0fQ.vXtGIg3jsZE38tuKGZfVZlDEcKPZdpdNo1ApIfhdIrI",
  "token_type": "Bearer"
}
```

d. Detail akun

My Collection > Post data

POST http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/auth/me

Auth Type: Bearer Token

Token: 2Jgwl-opOMCYIolRM5V_r37DU_j8Qkg

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: Cookies 1 Headers 10 Test Results 1/1

200 OK • 130 ms • 583 B

Save Response

```

1  {
2    "status": true,
3    "message": "User retrieved successfully",
4    "data": {
5      "id": "3",
6      "name": "Alya Mutiya Ningsih Pertiwi",
7      "email": "alyamutiya@gmail.com",
8      "created_at": "2026-05-02 15:55:20",
9      "updated_at": "2026-05-02 15:55:20"
10   }
11 }
```

e. Logout

The screenshot shows a Postman interface for a POST request. The URL is `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/auth/logout`. The request body is a JSON object with email and password fields. The response is a 200 OK status with a success message.

Request:

```
1 {
2   "email": "alyamutiya@gmail.com",
3   "password": "12345678"
4 }
```

Response:

```
1 {
2   "status": true,
3   "message": "Logout successful"
4 }
```

f. Create post

The screenshot shows a Postman interface for a POST request. The URL is `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post`. The request body is a JSON object with title, author, article, and image fields. The response is a 201 Created status with a success message and a data object containing post details.

Request:

Key	Value	Description	Bulk Edit
title	Text	Praktikum pekan 9	
author	Text	Alya Mutiya Ningsih Pertiwi	
article	Text	Modul 9: CodeIgniter 3 + HMVC + REST API + CR...	
image	File	TU-logogram-238x300.jpg	
Key	Text	Value	Description

Response:

```
1 {
2   "status": true,
3   "message": "Post created successfully",
4   "data": {
5     "id": "6",
6     "title": "Praktikum pekan 9",
7     "author": "Alya Mutiya Ningsih Pertiwi",
8     "article": "Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh",
9     "image": "post_1777713235_69f5c05306b3d.jpg",
10    "created_at": "2026-05-02 16:13:55",
11    "updated_at": "2026-05-02 16:13:55",
12    "image_url": "http://localhost/ci3_project/uploads/posts/post_1777713235_69f5c05306b3d.jpg"
13  }
14 }
```


g. Update post

The screenshot shows a Postman interface for a PUT request to `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post/6`. The request is configured with form-data. The body contains the following data:

Key	Value	Description	Bulk Edit
title	Praktikum pekan 9		
author	Alya Mutiya Ningsih Pertiwi		
article	Update Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh		
image	Logo_Telkom_University_potrait.png		

The response is a 200 OK status with a response time of 193 ms and a body size of 929 B. The response body is a JSON object:

```
1 {
2   "status": true,
3   "message": "Post updated successfully",
4   "data": {
5     "id": "6",
6     "title": "Praktikum pekan 9",
7     "author": "Alya Mutiya Ningsih Pertiwi",
8     "article": "Update Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh",
9     "image": "post_177713235_69f5c85386b3d.jpg",
10    "created_at": "2026-05-02 16:13:55",
11    "updated_at": "2026-05-02 16:26:49",
12    "image_url": "http://localhost/ci3_project/uploads/posts/post_177713235_69f5c85386b3d.jpg"
13  }
14 }
```

h. Delete post

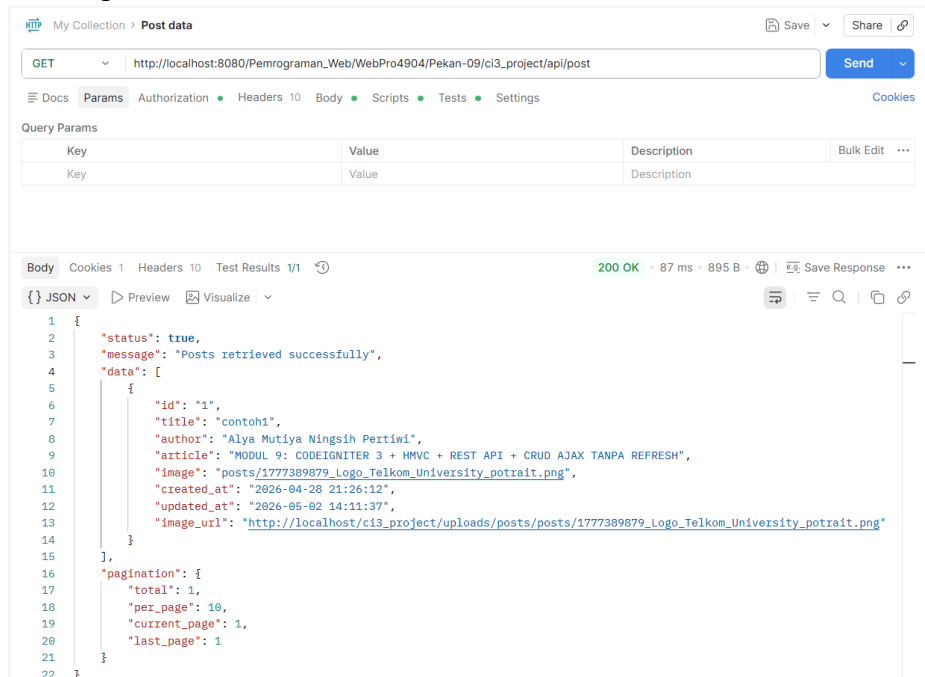
The screenshot shows a Postman interface for a DELETE request to `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post/6`. The request is configured with form-data. The body contains the following data:

Key	Value	Description	Bulk Edit
title	Praktikum pekan 9		
author	Alya Mutiya Ningsih Pertiwi		
article	Update Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh		
image	Logo_Telkom_University_potrait.png		

The response is a 200 OK status with a response time of 120 ms and a body size of 424 B. The response body is a JSON object:

```
1 {
2   "status": true,
3   "message": "Post deleted successfully"
4 }
```

i. Get all posts



My Collection > Post data

GET http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post Send

Params Authorization Headers 10 Body Scripts Tests Settings Cookies

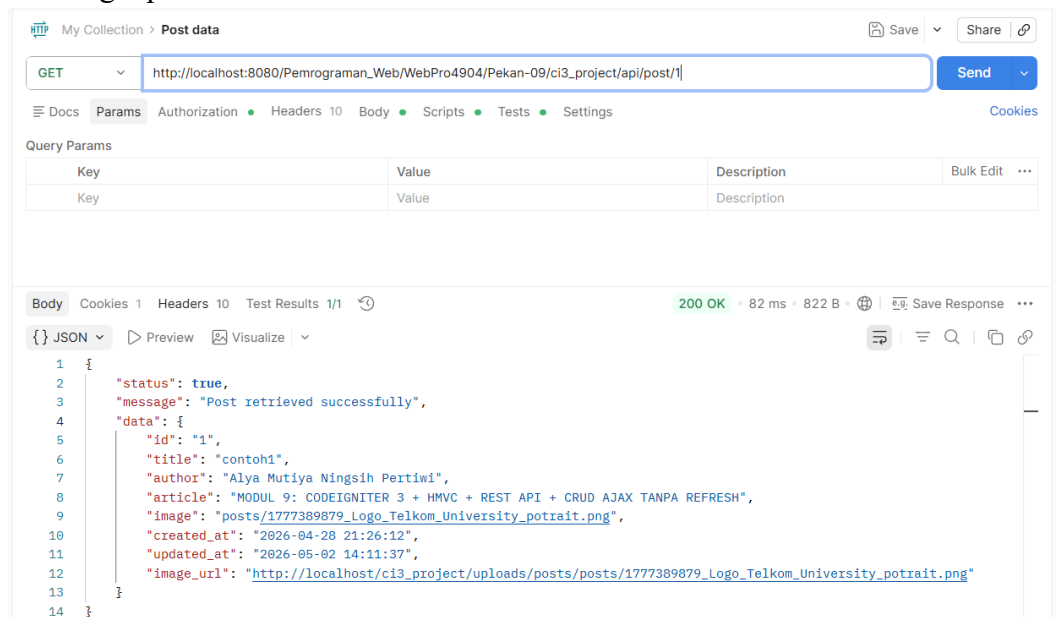
Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies 1 Headers 10 Test Results 1/1 200 OK • 87 ms • 895 B Save Response

```
{
  "status": true,
  "message": "Posts retrieved successfully",
  "data": [
    {
      "id": "1",
      "title": "contoh1",
      "author": "Alya Mutiya Ningsih Pertiwi",
      "article": "MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJAX TANPA REFRESH",
      "image": "posts/1777389879_Logo_Telkom_University_potrait.png",
      "created_at": "2026-04-28 21:26:12",
      "updated_at": "2026-05-02 14:11:37",
      "image_url": "http://localhost/ci3_project/uploads/posts/posts/1777389879_Logo_Telkom_University_potrait.png"
    }
  ],
  "pagination": {
    "total": 1,
    "per_page": 10,
    "current_page": 1,
    "last_page": 1
  }
}
```

j. Get single post



My Collection > Post data

GET http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post/1 Send

Params Authorization Headers 10 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies 1 Headers 10 Test Results 1/1 200 OK • 82 ms • 822 B Save Response

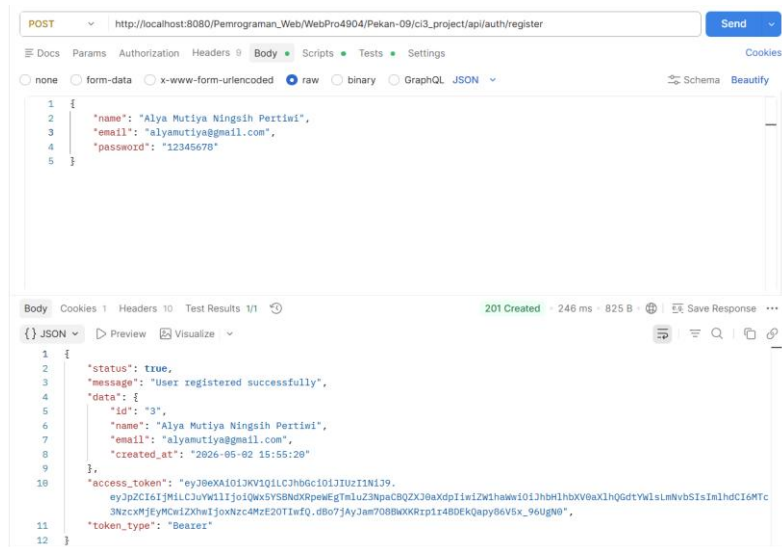
```
{
  "status": true,
  "message": "Post retrieved successfully",
  "data": {
    "id": "1",
    "title": "contoh1",
    "author": "Alya Mutiya Ningsih Pertiwi",
    "article": "MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJAX TANPA REFRESH",
    "image": "posts/1777389879_Logo_Telkom_University_potrait.png",
    "created_at": "2026-04-28 21:26:12",
    "updated_at": "2026-05-02 14:11:37",
    "image_url": "http://localhost/ci3_project/uploads/posts/posts/1777389879_Logo_Telkom_University_potrait.png"
  }
}
```

Alur pengujian:

1. Buka aplikasi Postman

-

- Contoh perintah:



The screenshot shows the Swagger UI interface. At the top, the URL bar displays the endpoint: `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post`. The 'POST' method is selected. Below the URL bar, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', 'Tests', and 'Settings'. The 'Body' tab is active, showing a table with the following content:

Key	Value	Description	Bulk Edit	...
☒ title	Text ▾	Praktikum pekan 9		
☒ author	Text ▾	Alya Mutiwa Ningsih Pertiwi		
☒ article	Text ▾	Modul 9: CodeIgniter 3 + HMVC + REST API + CR...		
☒ image	File ▾	TU-logogram-238x300.jpg	📎	
Key	Text ▾	Value		Description

Below the Swagger UI, the 'Body' tab is selected in the response section. It shows a JSON response with the following structure:

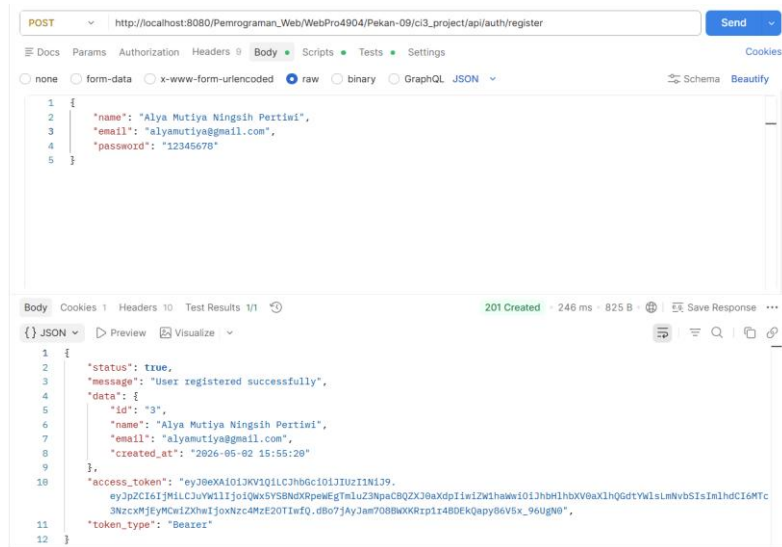
```

{
  "status": true,
  "message": "Post created successfully",
  "data": {
    "id": "6",
    "title": "Praktikum pekan 9",
    "author": "Alya Mutiwa Ningsih Pertiwi",
    "article": "Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh",
    "image": "post_1777713235_69f5c6386b3d.jpg",
    "created_at": "2026-05-02 16:13:55",
    "updated_at": "2026-05-02 16:13:55",
    "image_url": "http://localhost/ci3_project/uploads/posts/post_1777713235_69f5c6386b3d.jpg"
  }
}

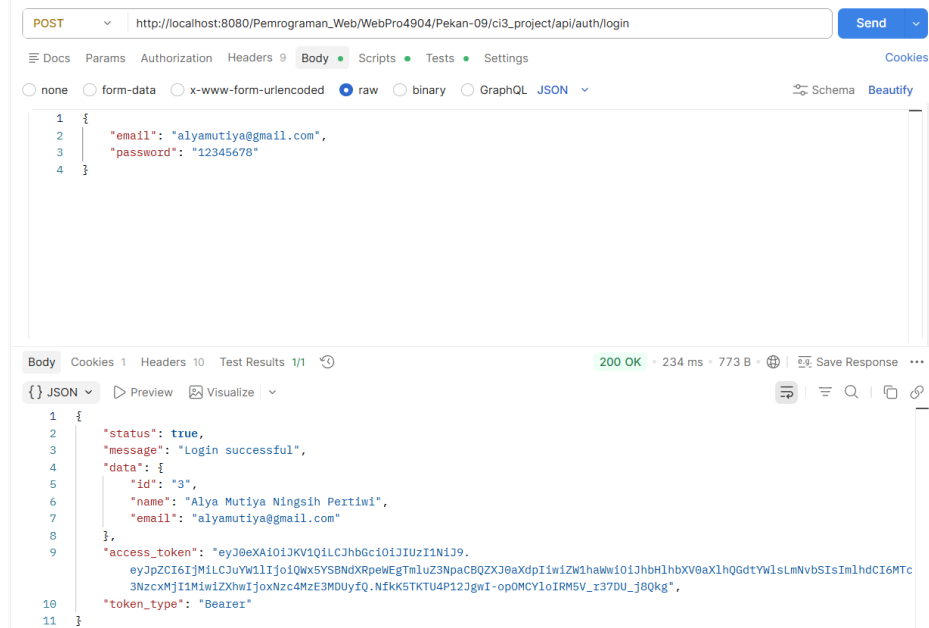
```

4. Hasil dan Dokumentasi Program

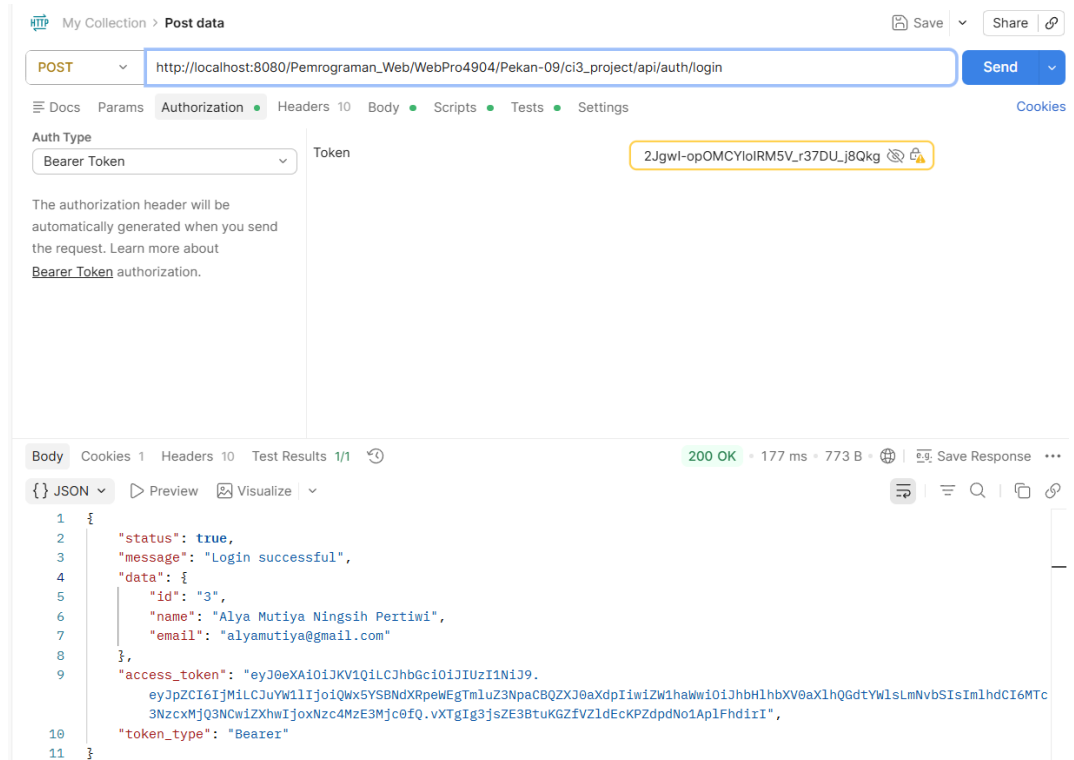
1. Register



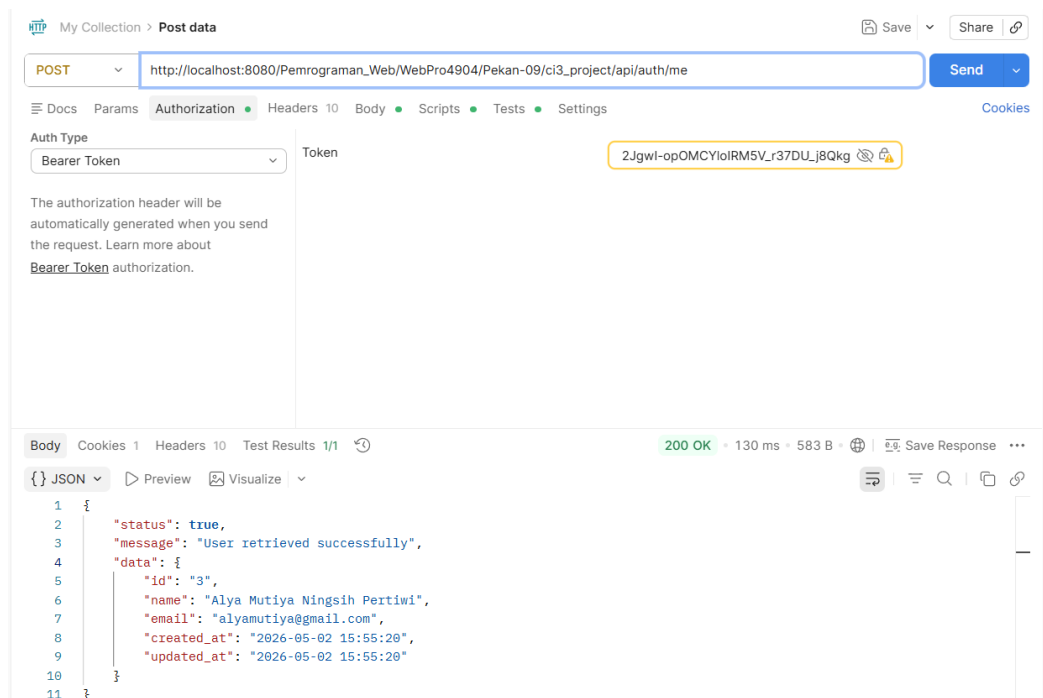
2. Login



3. Token keamanan login



4. Detail akun



5. Logout

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/auth/logout`
- Body (raw):**

```
1 {
2   "email": "alyamutiya@gmail.com",
3   "password": "12345678"
4 }
```
- Response (JSON):**

```
1 {
2   "status": true,
3   "message": "Logout successful"
4 }
```
- Status:** 200 OK, 112 ms, 416 B

6. Create post

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post`
- Body (form-data):**

Key	Value	Description
title	Praktikum pekan 9	
author	Alya Mutiya Ningsih Pertiwi	
article	Modul 9: CodeIgniter 3 + HMVC + REST API + CR...	
image	TU-logogram-238x300.jpg	
- Response (JSON):**

```
1 {
2   "status": true,
3   "message": "Post created successfully",
4   "data": {
5     "id": "6",
6     "title": "Praktikum pekan 9",
7     "author": "Alya Mutiya Ningsih Pertiwi",
8     "article": "Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh",
9     "image": "post_1777713235_69f5c05306b3d.jpg",
10    "created_at": "2026-05-02 16:13:55",
11    "updated_at": "2026-05-02 16:13:55",
12    "image_url": "http://localhost/ci3_project/uploads/posts/post_1777713235_69f5c05306b3d.jpg"
13  }
14 }
```
- Status:** 201 Created, 208 ms, 927 B

7. Update post

The screenshot shows a Postman interface for a PUT request to `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post/6`. The request body is in form-data with the following fields:

Key	Value	Description
title	Praktikum pekan 9	
author	Alya Mutiya Ningsih Pertiwi	
article	Update Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh	
image	Logo_Telkom_University_potrait.png	

The response is a 200 OK status with the following JSON body:

```
1 {
2   "status": true,
3   "message": "Post updated successfully",
4   "data": {
5     "id": "6",
6     "title": "Praktikum pekan 9",
7     "author": "Alya Mutiya Ningsih Pertiwi",
8     "article": "Update Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh",
9     "image": "post_177713235_69f5c85386b3d.jpg",
10    "created_at": "2026-05-02 16:13:55",
11    "updated_at": "2026-05-02 16:26:49",
12    "image_url": "http://localhost/ci3_project/uploads/posts/post_177713235_69f5c85386b3d.jpg"
13  }
14 }
```

8. Delete post

The screenshot shows a Postman interface for a DELETE request to `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post/6`. The request body is in form-data with the following fields:

Key	Value	Description
title	Praktikum pekan 9	
author	Alya Mutiya Ningsih Pertiwi	
article	Update Modul 9: CodeIgniter 3 + HMVC + REST API + CRUD AJAX Tanpa Refresh	
image	Logo_Telkom_University_potrait.png	

The response is a 200 OK status with the following JSON body:

```
1 {
2   "status": true,
3   "message": "Post deleted successfully"
4 }
```

9. Get all posts

The screenshot shows a Postman interface for a GET request to `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post`. The response is a 200 OK status with the following JSON body:

```
1 {
2   "status": true,
3   "message": "Posts retrieved successfully",
4   "data": [
5     {
6       "id": "1",
7       "title": "contoh1",
8       "author": "Alya Mutiya Ningsih Pertiwi",
9       "article": "MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJAX TANPA REFRESH",
10      "image": "posts/1777389879_Logo_Telkom_University_potrait.png",
11      "created_at": "2026-04-28 21:26:12",
12      "updated_at": "2026-05-02 14:11:37",
13      "image_url": "http://localhost/ci3_project/uploads/posts/posts/1777389879_Logo_Telkom_University_potrait.png"
14    },
15   ],
16   "pagination": {
17     "total": 1,
18     "per_page": 10,
19     "current_page": 1,
20     "last_page": 1
21   }
22 }
```

10. Get single post

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/Pemrograman_Web/WebPro4904/Pekan-09/ci3_project/api/post/1`. The response is a 200 OK status with a body containing JSON data:

```
{
  "status": true,
  "message": "Post retrieved successfully",
  "data": {
    "id": "1",
    "title": "contoh1",
    "author": "Alya Mutiya Ningsih Pertiwi",
    "article": "MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJAX TANPA REFRESH",
    "image": "posts/1777389879_Logo_Telkom_University_potrait.png",
    "created_at": "2026-04-28 21:26:12",
    "updated_at": "2026-05-02 14:11:37",
    "image_url": "http://localhost/ci3_project/uploads/posts/posts/1777389879_Logo_Telkom_University_potrait.png"
  }
}
```

11. Database user

The screenshot shows the phpMyAdmin interface for the 'users' table. The table contains two rows of data:

id	name	email	password	created_at	updated_at
2	Alya Mutiya Ningsih Pertiwi	alyamutiya0406@gmail.com	\$2y\$10\$4NAXlo4PdTF6EqBv0PxoqVFIAvO3u22IW4m.QrmG7L...	2026-05-02 13:39:45	2026-05-02 13:39:45
3	Alya Mutiya Ningsih Pertiwi	alyamutiya@gmail.com	\$2y\$10\$5vXrFX0Vsl0wba0ISH5nCeqyDaFKyIfV/S6sU3X3agZ...	2026-05-02 15:55:20	2026-05-02 15:55:20

12. Database post

The screenshot shows the phpMyAdmin interface for the 'posts' table. The table contains one row of data:

id	title	author	article	image	created_at	updated_at
1	contoh1	Alya Mutiya Ningsih Pertiwi	MODUL 9: CODEIGNITER 3 + HMVC + REST API + CRUD AJ...	posts/1777389879_Logo_Telkom_University_potrait.pn...	2026-04-28 21:26:12	2026-05-02 14:11:37

5. Analisis dan Pembahasan

Pada praktikum Pekan 9 ini, aplikasi dikembangkan menggunakan CodeIgniter 3 dengan pendekatan HMVC, di mana setiap fitur dipisahkan ke dalam modul seperti

Crudjs dan Api. Struktur ini membuat kode lebih rapi, terorganisir, dan mudah dikembangkan.

Selain itu, diterapkan REST API untuk menghubungkan frontend dan backend menggunakan format JSON. Proses seperti login, pengambilan data, hingga CRUD dilakukan melalui metode HTTP (GET, POST, PUT, DELETE) dan dilengkapi dengan keamanan token.

Penggunaan AJAX memungkinkan proses CRUD berjalan tanpa reload halaman, sehingga aplikasi menjadi lebih responsif. Pengujian menggunakan Postman menunjukkan bahwa seluruh endpoint API berjalan dengan baik.

6. Kesimpulan

Praktikum ini menunjukkan bahwa penggunaan HMVC, REST API, dan AJAX pada CodeIgniter 3 mampu menghasilkan aplikasi yang lebih terstruktur, fleksibel, dan interaktif. Sistem menjadi lebih mudah dikembangkan serta memberikan pengalaman pengguna yang lebih baik.

7. Lampiran berisi Link repositori GitHub

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/AlyaMutiyaNingsihPertiwi-707012500132>